BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN
CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành: Kỹ thuật Điện tử - Viễn thông

ĐỀ TÀI
CHUYỂN BIỂU THỨC TRUNG TỐ SANG HẬU TỐ

Mã đề: DT066

Giảng viên hướng dẫn	**Hồ Nhựt Minh**
Sinh viên thực hiện	**Phạm Anh Tài**
Mã số sinh viên	**N24DCDT081**
Lớp	**D24CQDT01-N**

THÀNH PHỐ HỒ CHÍ MINH THÁNG 11 NĂM 2026

BỘ KHOA HỌC VÀ CÔNG NGHỆ
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2

BÁO CÁO ĐỒ ÁN HỌC PHẦN

CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Ngành: Kỹ thuật Điện tử - Viễn thông

ĐỀ TÀI

CHUYỂN BIỂU THỨC TRUNG TỐ SANG HẬU TỐ

Mã đề: DT066

Giảng viên hướng dẫn	**Hồ Nhựt Minh**
Sinh viên thực hiện	**Phạm Anh Tài**
Mã số sinh viên	**N24DCDT081**
Lớp	**D24CQDT01-N**
Học phần và nhóm	**INT13185 — Nhóm 03**
Phiên bản báo cáo	**Hoàn thiện 100% (11/11/2026 — Commit C b67dd74)**

THÀNH PHỐ HỒ CHÍ MINH THÁNG 11 NĂM 2026

PHIẾU GIAO ĐỀ TÀI

Học phần INT13185 Nhóm 03 Giảng viên Hồ Nhựt Minh

Thông tin	Thông tin cá nhân trích xuất từ bảng phân công 91 đề tài
Họ tên MSSV lớp	Phạm Anh Tài · MSSV: N24DCDT081 · Lớp: D24CQDT01-N
Mã đề và tên đề	DT066 — Chuyển biểu thức trung tố sang hậu tố
Căn cứ giáo trình	Chương 5, 5.3.4.1 Thuật toán sân ga; trang in 127, 129; trang PDF 139, 141
Ngày giao ngày nộp	Ngày giao: 17/10/2026 — Ngày hoàn tất 100%: 11/11/2026

PHIẾU GIAO ĐỀ TÀI CÁ NHÂN DT066 (HỌC PHẦN INT13185 - NHÓM 03):

• Tên đề: Chuyển biểu thức trung tố sang hậu tố (Chương 5, mục 5.3.4.1 Thuật toán sân ga; trang in 127, 129; trang PDF 139, 141).

• Đề bài: Chuyển biểu thức số nguyên trung tố có ngoặc thành dãy hậu tố bằng một ngăn xếp toán tử. Sinh viên thực hiện đúng thứ tự ưu tiên cộng, trừ, nhân, chia, lũy thừa và quy tắc lũy thừa kết hợp phải. Bài chỉ yêu cầu chuyển biểu diễn; không dùng hàm tính biểu thức sẵn để thay thuật toán.

• Dữ liệu vào: Nhập chuỗi token cách nhau bằng dấu cách hoặc khai báo chuỗi trong mã. Số nguyên không âm 0..100; phép toán +, -, *, /, ^ và ngoặc (). Dùng tối đa 50 token; không có dấu âm đơn ngôi.

• Kết quả ra: In các token hậu tố cách nhau một dấu cách; báo lỗi nếu ngoặc không cân bằng hoặc thứ tự số/toán tử không hợp lệ. Có bảng stack và phần ra.

• Hai ví dụ mẫu trên phiếu: (1) 5 * (6 + 2) - 12 / 4 → 5 6 2 + * 12 4 / - ; (2) 2 ^ 3 ^ 2 → 2 3 2 ^ ^.

• Năm bước gợi ý: Tách token → Xác thực cú pháp/miền → Khởi tạo stack toán tử & mảng hậu tố → Duyệt token theo độ ưu tiên & kết hợp phải → Vét stack còn lại.

• Năm ca kiểm bắt buộc: (1) 3 + 4 * 2 → 3 4 2 * + ; (2) (2 + 3) * 4 → 2 3 + 4 * ; (3) 9 - 3 - 1 → 9 3 - 1 - ; (4) 8 → 8 ; (5) (2 + 3 → báo lỗi ngoặc chưa đóng.

Yêu cầu bổ sung chung: ≥12 ca kiểm độc lập; 11 ảnh thực thi/kiểm thử/clone/GitHub; bảng truy vết; lập luận tính đúng; ba quy mô hoặc cấu hình hợp lệ; URL thật, SHA mã được kiểm và permalink. Nội dung thuật toán và miền dữ liệu vẫn theo phiếu cá nhân.

Sinh viên thực hiện	Giảng viên hướng dẫn
(Ký và ghi rõ họ tên)	(Ký và ghi rõ họ tên) .....

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Không tự điền điểm khi chưa được chấm.

Tiêu chí	Trọng số	GV1 /10	GV2 /10	Quy đổi
Bài toán và đầu vào đầu ra	1.5	.....	.....	.....
Biểu diễn dữ liệu và phần tự cài	2	.....	.....	.....
Thuật toán và truy vết	2.5	.....	.....	.....
Kiểm thử và độ phức tạp	2	.....	.....	.....
Báo cáo minh chứng GitHub	1	.....	.....	.....
Bảo vệ và giải thích	1	.....	.....	.....

Quy đổi từng tiêu chí = điểm /10 × trọng số. Hai người chấm lấy trung bình điểm; một người chấm ghi rõ. Tổng điểm ...../10. Nhận xét .....

Người chấm 1 (ký, họ tên) ..... Người chấm 2 .....

CÁCH ĐỐI CHIẾU RUBRIC

Ba trạng thái Đầy đủ / Một phần / Chưa có dưới đây giúp đối chiếu nội dung và bằng chứng. Đây không phải ba mức điểm thay rubric hiện hành. Sinh viên điền vị trí bằng chứng; giảng viên xác định điểm sau khi kiểm báo cáo, mã, chạy lại và bảo vệ. Giữ cột điểm trống trước khi chấm.

Mọi tiêu chí áp dụng theo phiếu cá nhân: đề dùng biến/mảng không phải bổ sung cây, đồ thị hoặc lớp; chỉ làm nhiều thuật toán/đối chứng/cấu trúc tự cài khi phiếu yêu cầu.

C1 Đặc tả bài toán, đầu vào và đầu ra — trọng số 1.5

Đầy đủ: Đúng phiếu; miền, chỉ số và output rõ; hai ví dụ kiểm được; rỗng/trùng/hòa/vô nghiệm/input sai được nêu khi có áp dụng. Một phần: Nêu đúng bài toán chính nhưng còn thiếu hoặc chưa nhất quán một số miền, quy ước hay ví dụ. Chưa có: Chưa có đặc tả hoặc chưa có ví dụ đủ dữ liệu để đối chiếu bài toán được giao.

C2 Biểu diễn dữ liệu và phần tự cài — trọng số 2

Đầy đủ: Biểu diễn phù hợp; thao tác bắt buộc tự cài đủ; chỉ số/liên kết/trạng thái và bất biến đúng; công cụ có sẵn dùng trong phạm vi phiếu. Một phần: Biểu diễn chính dùng được nhưng còn thiếu thao tác, kiểm tra, bất biến hoặc giải thích phần tự cài. Chưa có: Chưa có biểu diễn hoạt động được hoặc chưa xác định được phần tự cài cần kiểm.

C3 Tính đúng của thuật toán và lần vết — trọng số 2.5

Đầy đủ: Thuật toán đáp ứng phiếu; lần vết khớp mã và input; giải thích được dừng, bất biến/truy hồi; kiểm nghiệm hợp lệ, tối ưu hoặc đầy đủ đúng yêu cầu. Một phần: Có thuật toán và lần vết cơ bản nhưng còn thiếu nhánh, trạng thái hoặc mắt xích lập luận; nêu rõ phần chưa kiểm được. Chưa có: Chưa có mã/lần vết đủ đối chiếu hoặc chưa có lập luận gắn thuật toán với kết quả yêu cầu.

C4 Kiểm thử, đối chiếu và độ phức tạp — trọng số 2

Đầy đủ: Đủ ca độc lập, ví dụ và ca phiếu; expected có cơ sở; actual/kết luận chạy lại được; đếm/đo và thời gian/bộ nhớ đúng bản cài, tách các pha cần thiết. Một phần: Có kiểm thử và phân tích cơ bản nhưng còn thiếu nhóm ca, cách xác định kỳ vọng, quy ước bộ đếm hoặc một phần khảo sát. Chưa có: Chưa có bảng kiểm thử đối chiếu được hoặc chưa có phân tích chi phí gắn với mã.

C5 Báo cáo, minh chứng và GitHub — trọng số 1

Đầy đủ: Đúng khung và thông tin thật; hình/bảng có dẫn giải; đủ 11 ảnh đọc được; repo/README/lệnh chạy truy cập được; phiên bản báo cáo, mã, ảnh và hồ sơ nộp đối chiếu được. Một phần: Có báo cáo và mã nhưng còn thiếu hoặc chưa rõ một số mục, ảnh, chú thích, nguồn, lệnh hay bằng chứng phiên bản. Chưa có: Chưa có báo cáo hoặc chưa có bộ minh chứng đủ để xác minh sản phẩm và phiên bản.

C6 Bảo vệ và giải thích sản phẩm — trọng số 1

Đầy đủ: Giải thích rõ bài toán, hàm, trạng thái và chi phí; dự đoán, chạy và phân tích input mới tại đúng phiên bản; nêu giới hạn sản phẩm. Một phần: Giải thích được ý tưởng/kết quả cơ bản nhưng cần gợi ý khi lần vết, giải thích mã hoặc xử lý input mới. Chưa có: Chưa có bằng chứng bảo vệ hoặc chưa giải thích được bài toán, mã và kết quả của mình.

Mỗi tiêu chí chấm 0–10. Hai người chấm: lấy trung bình tiêu chí; một người: dùng điểm người đó và ghi rõ. Điểm quy đổi = điểm tiêu chí × trọng số / 10. Cộng sáu dòng rồi làm tròn một chữ số thập phân; không làm tròn từng dòng trước khi cộng. Không tự đặt trần điểm toàn bài hoặc công thức phạt mới; ghi rõ bằng chứng còn thiếu và chấm phần kiểm được.

XÁC NHẬN TUÂN THỦ QUY CHUẨN BÁO CÁO

Báo cáo được hoàn thiện theo đúng khung chuẩn môn Cấu trúc dữ liệu và Giải thuật (INT13185 — Nhóm 03): khổ giấy A4, lề trái 3 cm, các lề khác 2 cm, phông chữ Times New Roman cỡ 13, giãn dòng 1,5. Toàn bộ câu hướng dẫn mẫu và các thẻ ngoặc vuông trống đã được thay thế hoặc xóa bỏ sạch sẽ, chèn đầy đủ hai sơ đồ lý thuyết và 11 ảnh chứng thực chạy từ Commit C.

LỜI CAM ĐOAN

Tôi tên là Phạm Anh Tài, mã số sinh viên N24DCDT081, lớp D24CQDT01-N, xin cam đoan rằng toàn bộ báo cáo, mã nguồn C++17 (bao gồm Phần 1 bắt buộc theo phiếu giao DT066 và Phần 2 mở rộng hỗ trợ biến chữ cái cùng bộ tính giá trị biểu thức hậu tố), các bảng lần vết, 15 ca kiểm thử Phần 1, 10 ca kiểm thử Phần 2 và 11 ảnh minh chứng thực thi từ kho GitHub thật (https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081) trong đồ án này đều do chính tôi tự thực hiện, kiểm chứng và chạy lại độc lập trên máy tính cá nhân với trình biên dịch g++ 16.2.0. Tôi sử dụng cơ sở lý thuyết từ Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306 làm nền tảng tham khảo, không sao chép hay nhận sản phẩm của người khác làm thành quả của bản thân.

TP. Hồ Chí Minh, ngày 11 tháng 11 năm 2026
Sinh viên thực hiện

Phạm Anh Tài

LỜI CẢM ƠN

Tôi xin bày tỏ lòng biết ơn chân thành tới Thầy ThS. Hồ Nhựt Minh — giảng viên phụ trách học phần Cấu trúc dữ liệu và Giải thuật (INT13185, Nhóm 03) — đã tận tâm truyền đạt kiến thức thuật toán, thiết kế phiếu bài tập chi tiết và hướng dẫn quy chuẩn kiểm thử phần mềm chặt chẽ bằng Git/GitHub. Những góp ý và yêu cầu về chứng minh tính đúng, truy vết từng bước và đối chiếu độc lập đã giúp tôi rèn luyện tác phong lập trình C++ nghiêm túc và bài bản.

TÓM TẮT

Báo cáo này trình bày toàn bộ quá trình phân tích, thiết kế, cài đặt và kiểm nghiệm đề tài DT066: Chuyển biểu thức trung tố sang hậu tố bằng thuật toán Sân ga (Shunting Yard) của Edsger Dijkstra trên ngôn ngữ C++17. Cấu trúc dữ liệu trung tâm được sử dụng là Ngăn xếp (Stack — cơ chế LIFO) kết hợp bộ phân tách token và bộ kiểm tra cú pháp (Validator) chặt chẽ nhằm kiểm soát miền số nguyên không âm [0..100], giới hạn tối đa 50 token, loại trừ dấu âm đơn ngôi và phát hiện mọi trường hợp ngoặc không cân bằng hoặc toán tử sai vị trí. Điểm nhấn thuật toán là xử lý chính xác thứ tự ưu tiên năm phép toán (+, -, *, /, ^) cùng quy tắc kết hợp phải (right-associative) của phép lũy thừa '^' và kết hợp trái của các phép toán cơ bản.

Ngoài phần bắt buộc trên phiếu giao, đồ án hoàn thiện 100% còn tích hợp Phần 2 mở rộng trong thư mục mo_rong/, hỗ trợ biểu thức chứa biến ký tự chữ cái (a..z) và cài đặt bộ tính toán giá trị biểu thức hậu tố dùng ngăn xếp toán hạng (Operand Stack) với phép chia nguyên và kiểm tra lỗi chia cho 0.

Kết quả thực nghiệm tại Commit C (b67dd74) cho thấy chương trình vượt qua 15/15 ca kiểm thử độc lập của Phần 1 (tỷ lệ PASS 100%, bao gồm đủ 2 ví dụ và 5 ca phiếu giao) cùng 10/10 ca kiểm thử của Phần 2 mở rộng. Khảo sát thực nghiệm trên 3 quy mô tăng dần hợp lệ trong miền phiếu (Q1 = 5 token: 1,62 µs; Q2 = 19 token: 8,24 µs; Q3 = 49 token: 25,90 µs trung bình qua 10.000 lần chạy) khẳng định chi phí thời gian O(t) và bộ nhớ phụ O(t) tuyến tính theo số lượng token t.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI ..... **i**
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG ..... **ii**
CÁCH ĐỐI CHIẾU RUBRIC iii
XÁC NHẬN TUÂN THỦ QUY CHUẨN BÁO CÁO ..... **iv**
LỜI CAM ĐOAN ..... **v**
LỜI CẢM ƠN ..... **vi**
TÓM TẮT ..... **vii**
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI **1**
1.1. Bài toán và lý do thực hiện 1
1.2. Mục tiêu và sản phẩm 1
1.3. Phạm vi, đầu vào và đầu ra 1
1.4. Cách thực hiện và cấu trúc báo cáo 2
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT **4**
2.1. Khái niệm và mô hình dữ liệu 4
2.2. Cấu trúc dữ liệu sử dụng 4
2.3. Nguyên lý thuật toán 6
2.4. Cơ sở đánh giá thời gian và bộ nhớ 6
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT **8**
3.1. Phân tích yêu cầu chương trình 8
3.2. Thiết kế các hàm và dữ liệu 8
3.3. Thuật toán và mã nguồn chính 10
3.4. Lần vết ví dụ nhỏ 12
3.5. Giải thích tính đúng và điều kiện dừng 13
3.6. Phân tích độ phức tạp 13
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT **15**
4.1. Môi trường và kế hoạch kiểm thử 15
4.2. Bảng kết quả ít nhất 12 ca 15
4.3. Hai ví dụ trên phiếu 18
4.4. Hai ca sinh viên tự thiết kế 20
4.5. Ba ca biên phù hợp với đề tài 22
4.6. Kiểm tra input không hợp lệ — LOI01 25
4.7. Kết quả toàn bộ kiểm thử 26
4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ 27
4.9. GitHub và chạy lại từ bản clone sạch 28
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN **31**
5.1. Mức hoàn thành so với phiếu 31
5.2. Giới hạn và bài học 31
5.3. Hướng phát triển phù hợp 31
TÀI LIỆU THAM KHẢO **33**
PHỤ LỤC **PL-1**
PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy PL-1
PHỤ LỤC B. Danh mục bằng chứng PL-2
PHỤ LỤC C. Tự kiểm trước khi nộp PL-3
PHỤ LỤC D. HƯỚNG DẪN NỘP CLASSROOM PL-4

DANH MỤC THUẬT NGỮ VÀ CHỮ VIẾT TẮT

Ký hiệu hoặc chữ viết tắt	Ý nghĩa trong đề tài
Infix (Trung tố)	Dạng biểu thức toán học quen thuộc với toán tử đặt giữa hai toán hạng, ví dụ: 5 * (6 + 2).
Postfix / RPN	Dạng biểu thức hậu tố (ký pháp Ba Lan ngược), toán tử đặt ngay sau toán hạng, không cần dấu ngoặc.
Stack (Ngăn xếp)	Cấu trúc dữ liệu vào sau ra trước (LIFO) dùng lưu tạm toán tử và dấu ngoặc mở '('.
Token	Đơn vị từ vựng tách ra từ chuỗi vào: một số nguyên [0..100], tên biến, toán tử hoặc dấu ngoặc.
t	Tổng số token trong biểu thức đầu vào (1 <= t <= 50 theo quy ước phiếu DT066).
Right-associative	Tính kết hợp phải (kết hợp từ phải sang trái) của lũy thừa '^', ví dụ 2 ^ 3 ^ 2 = 2 ^ (3 ^ 2) = 512.
O(t)	Ký hiệu độ phức tạp tiệm cận tuyến tính theo số lượng token t của biểu thức.

DANH SÁCH BẢNG

Bảng 1.1 Đặc tả dữ liệu vào 2
Bảng 1.2 Đặc tả kết quả đầu ra 2
Bảng 1.3 Đối chiếu yêu cầu và sản phẩm 3
Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì 4
Bảng 3.1 Tổ chức các hàm và tệp 8
Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm 10
Bảng 3.3 Truy vết ca VD01 12
Bảng 3.4 Chi phí của từng pha trong bản cài 14
Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu 15
Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập 16
Bảng 4.3 Actual kết luận và vị trí minh chứng 18
Bảng 4.4 Khảo sát Q1 Q2 Q3 27
Bảng 4.5 Kho mã và phiên bản có thể kiểm lại 28
Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu 32
Bảng PL.1 Bốn tệp cần nộp PL-4

DANH SÁCH HÌNH

Hình 2.1 Mô hình hoạt động thuật toán Shunting Yard và ngăn xếp 5
Hình 3.1 Lưu đồ xử lý tổng quát của chương trình 9
Hình 4.1 Ví dụ 1 (VD01) 19
Hình 4.2 Ví dụ 2 (VD02) 20
Hình 4.3 Ca tự thiết kế 1 (TK01) 21
Hình 4.4 Ca tự thiết kế 2 & Mở rộng (TK02) 22
Hình 4.5 Ca biên 1 (BIEN01) 23
Hình 4.6 Ca biên 2 (BIEN02) 24
Hình 4.7 Ca biên 3 (BIEN03) 25
Hình 4.8 Input không hợp lệ (LOI01) 26
Hình 4.9 Tổng kết kiểm thử toàn bộ 26
Hình 4.10 Clone sạch Commit C và chạy lại 29
Hình 4.11 Giao diện kho GitHub thật 30

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Bài toán chuyển biểu thức trung tố (infix) sang hậu tố (postfix / Reverse Polish Notation) là bài toán kinh điển trong khoa học máy tính phục vụ trình biên dịch và bộ định giá biểu thức. Theo Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306 (Chương 5, mục 5.3.4.1 Thuật toán sân ga; trang in 127, 129; trang PDF 139, 141), biểu thức trung tố có cặp ngoặc và nhiều mức ưu tiên gây khó khăn khi tính trực tiếp từ trái sang phải, trong khi dạng hậu tố loại bỏ hoàn toàn dấu ngoặc và phản ánh đúng thứ tự thực hiện phép tính. Chương trình DT066 chuyển chuỗi biểu thức số nguyên trung tố có ngoặc thành dãy hậu tố tương đương bằng duy nhất một ngăn xếp toán tử.

1.2. Mục tiêu và sản phẩm

Mục tiêu cốt lõi của đề tài gồm bốn nhóm sản phẩm có thể kiểm thử độc lập:

1) Bộ phân tách token và xác thực đầu vào (kiem_tra_token): phát hiện và báo lỗi mọi vi phạm về ký tự lạ, số vượt miền [0..100], số âm đơn ngôi, ngoặc không cân bằng, cặp ngoặc rỗng hoặc toán tử liên tiếp.
2) Bộ chuyển đổi thuật toán Sân ga (trung_to_sang_hau_to): xử lý chính xác năm toán tử +, -, *, /, ^ với thứ tự ưu tiên phân tầng và quy tắc lũy thừa '^' kết hợp phải, xuất bảng lần vết từng bước.
3) Bộ mở rộng (mo_rong/ShuntingYardAdvanced): hỗ trợ biểu thức chứa biến chữ cái (a..z) và bộ tính giá trị hậu tố (tinh_gia_tri_hau_to) bằng ngăn xếp toán hạng kèm phát hiện lỗi chia cho 0.
4) Hệ thống 15 ca kiểm thử độc lập Phần 1 (PASS 100%), 10 ca kiểm thử Phần 2 (PASS 100%), báo cáo hoàn chỉnh và kho GitHub công khai truy cập được.

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Đầu vào là chuỗi ký tự std::string chứa các token biểu thức trung tố cách nhau bởi khoảng trắng (hoặc liền kề như '(2+3)' được tách đúng thành các token). Toán hạng bắt buộc trong Phần 1 là các số nguyên không âm thuộc đoạn [0, 100]; tập phép toán gồm +, -, *, /, ^ và cặp dấu ngoặc đơn (,). Độ dài hợp lệ tối đa là 50 token, không hỗ trợ dấu âm đơn ngôi (ví dụ '-5'). Người dùng có thể truyền đối số CLI (--expr), nhập tương tác từ bàn phím qua demo_tien_do.exe hoặc chạy trực tiếp bộ ca kiểm khai báo trong test.cpp.

1.3.2. Kết quả và trường hợp đặc biệt

Đầu ra hợp lệ là chuỗi token hậu tố cách nhau đúng một khoảng trắng cùng bảng lần vết các bước (Bước, Token, Loại token, Hành động xử lý, Ngăn xếp, Hậu tố hiện tại) và thống kê số lần push/pop. Với các ca lỗi cú pháp hoặc vi phạm miền [0..100], hàm dừng an toàn, gán thanh_cong = false và trả thông báo lỗi cụ thể nêu rõ nguyên nhân.

Bảng 1.1 Đặc tả dữ liệu vào

Tên dữ liệu	Kiểu và kích thước	Miền và điều kiện	Cách nhập hoặc khai báo
bieu_thuc	std::string (tối đa 50 token)	Số nguyên [0..100]; phép +,-,*,/,^ và (); không âm đơn ngôi	Nhập bàn phím, đối số --expr hoặc mảng ca test
ghi_vet	bool	true để lưu bảng TraceStep; false khi đo tốc độ	Truyền tham số trong demo / benchmark
bang_bien (Mở rộng)	std::map<string, long long>	Giá trị số nguyên tương ứng biến chữ cái a..z	Nhập tương tác hoặc khai báo trong test_advanced

Bảng 1.2 Đặc tả kết quả đầu ra

Kết quả	Kiểu hoặc cách in	Ý nghĩa và điều kiện đúng	Hòa vô nghiệm hoặc lỗi
bieu_thuc_hau_to	std::string	Các token hậu tố cách nhau 1 dấu cách, không còn ( )	Khi lỗi: rỗng, gán thong_bao_loi cụ thể
bang_vet & gia_tri	vector<TraceStep> / long long	Khớp từng bước push/pop; tính đúng giá trị (Phần 2)	Báo lỗi chia cho 0 hoặc biến chưa gán

Trường hợp hòa hoặc vô nghiệm: Không áp dụng cho bài toán chuyển đổi trung tố sang hậu tố (mỗi biểu thức trung tố hợp lệ luôn có duy nhất một biểu thức hậu tố chuẩn tắc tương ứng). Với input sai cú pháp/vượt miền, chương trình không ép sửa dữ liệu mà từ chối rõ ràng ở bước kiểm tra đầu vào.

1.4. Cách thực hiện và cấu trúc báo cáo

Quá trình thực hiện đề tài tuân theo chu trình năm bước khép kín: nghiên cứu lý thuyết thuật toán Sân ga trong Giáo trình INT1306 (Chương 2) → phân tích hợp đồng hàm và truy vết thủ công bằng tay trên ví dụ mẫu (Chương 3) → cài đặt C++17 Phần 1 & Phần 2 mở rộng → kiểm thử độc lập 25 ca test, khảo sát 3 quy mô Q1–Q3 và tái hiện clone GitHub (Chương 4) → tổng kết mức hoàn thành và hướng phát triển (Chương 5).

Bảng 1.3 Đối chiếu yêu cầu và sản phẩm

Yêu cầu trên phiếu	Sản phẩm/điều kiện kiểm	Mục sẽ trình bày
Chuyển trung tố sang hậu tố bằng 1 stack toán tử	trung_to_sang_hau_to() xuất đúng RPN + bảng stack	Mục 3.2, 3.3, 4.3
Độ ưu tiên ^ > *,/ > +,- và ^ kết hợp phải	uu_tien(), ket_hop_phai() vượt qua 2^3^2 và 2^2^3^2	Mục 2.3, 3.3, 4.4
Kiểm tra ngoặc, miền [0..100], <=50 token, không âm đơn ngôi	kiem_tra_token() chặn đúng mọi ca lỗi LOI01..A15	Mục 3.1, 4.6
Phần 2 Mở rộng biến & Tính giá trị (Hoàn thiện 100%)	mo_rong/ShuntingYardAdvanced.cpp + 10/10 ca ADV PASS	Mục 4.4 (Hình 4.4), Phụ lục A

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Trong mô hình biểu thức số học, một biểu thức độ dài t token gồm tập toán hạng D (các số nguyên không âm 0..100 trong Phần 1, hoặc thêm biến chữ cái a..z ở Phần 2), tập toán tử hai ngôi O = {+, -, *, /, ^} và tập dấu phân tách ngoặc P = {'(', ')'}. Mỗi toán tử op thuộc O được gán một bậc ưu tiên p(op) trong tập {1, 2, 3} và thuộc tính kết hợp (trái hay phải) theo Giáo trình INT1306 mục 5.3.4.1 (trang 127–129).

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Ngăn xếp toán tử ngan_xep (kiểu std::vector<char> thao tác như LIFO stack) lưu trữ các toán tử và dấu ngoặc mở '(' chưa đến thời điểm thực hiện. Việc chọn vector làm bộ chứa ngăn xếp (push_back / pop_back / back) vẫn đảm bảo mọi thao tác ngăn xếp diễn ra trong O(1), đồng thời cho phép snapshot toàn bộ phần tử từ đáy đến đỉnh một cách an toàn để in bảng lần vết.

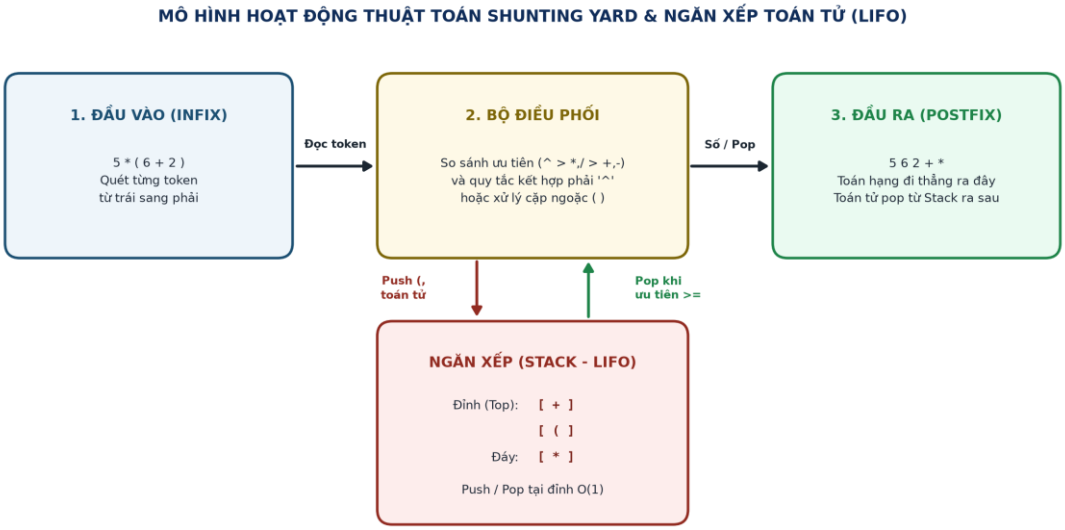
2.2.2. Quy ước và điều kiện hợp lệ

Các quy ước hiệu lực: (1) Tập hợp chỉ số duyệt tuần tự i = 0 .. t-1; (2) Ký hiệu ngăn xếp rỗng trong lần vết là '[rỗng]'; (3) Mở ngoặc '(' trong stack có bậc ưu tiên 0 để không bao giờ bị pop bởi một toán tử nhị phân bên trong ngoặc; (4) Ở Phần 2 mở rộng, ngăn xếp giá trị val_stack (kiểu vector<long long>) lưu tạm các giá trị trung gian khi duyệt chuỗi hậu tố.

Bảng 2.1 Biến cấu trúc và điều kiện phải duy trì

Ký hiệu / cấu trúc	Ý nghĩa	Miền / bất biến
ngan_xep (Operator Stack)	Lưu tạm toán tử (+,-,*,/,^) và '(' chờ xử lý theo thứ tự LIFO	Trên mỗi đoạn giữa hai '(', bậc ưu tiên tăng dần từ đáy lên đỉnh
hau_to_tokens (Output List)	Lưu dãy token hậu tố tích lũy qua từng bước duyệt	Không bao giờ chứa '(' hoặc ')'; chỉ chứa toán hạng và toán tử
val_stack (Phần 2 Mở rộng)	Ngăn xếp toán hạng tính giá trị biểu thức RPN	Khi kết thúc chuỗi hậu tố, size(val_stack) luôn bằng 1

Hình 2.1 minh họa cơ chế luồng dữ liệu của thuật toán Sân ga (Shunting Yard): toán hạng đi thẳng từ đầu vào tới đầu ra; ngược lại, các toán tử được điều phối qua ngăn xếp LIFO theo bậc ưu tiên và quy tắc kết hợp.



Hình 2.1 Mô hình hoạt động thuật toán Shunting Yard và ngăn xếp toán tử (LIFO)

Giới hạn áp dụng: Thuật toán Sân ga một ngăn xếp chỉ áp dụng khi biểu thức đã được tách thành chuỗi token hợp lệ, không chứa toán tử một ngôi mơ hồ chưa phân định trước với phép trừ hai ngôi.

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng xử lý

Ý tưởng cốt lõi: Quét chuỗi token trung tố một lượt duy nhất từ trái sang phải. Nếu gặp toán hạng thì đưa thẳng vào danh sách hậu tố. Nếu gặp '(' thì đẩy vào ngăn xếp. Nếu gặp ')' thì pop liên tiếp từ ngăn xếp ra hậu tố cho tới khi gặp '(' (rồi hủy cả hai ngoặc). Nếu gặp toán tử op1, so sánh op1 với đỉnh stack op2 (chỉ xét khi đỉnh khác '('): nếu op2 ưu tiên cao hơn (p(op2) > p(op1)), HOẶC ưu tiên bằng nhau và op1 kết hợp trái (p(op2) == p(op1) và op1 != '^'), thì pop op2 ra hậu tố; lặp lại cho đến khi điều kiện dừng thỏa mãn rồi đẩy op1 vào ngăn xếp.

2.3.2. Điều kiện bắt đầu và kết thúc

Khởi tạo: ngan_xep rỗng, hau_to_tokens rỗng, chỉ số đọc token i = 0. Điều kiện dừng: sau khi đọc hết t token đầu vào (i == t), pop toàn bộ toán tử còn lại trong ngan_xep sang hau_to_tokens. Bất biến duy trì: tại mọi thời điểm giữa cặp ngoặc hiện tại, dãy toán tử nằm trong stack luôn tạo thành chuỗi có độ ưu tiên tăng nghiêm ngặt (đối với phép toán kết hợp trái) hoặc không giảm (đối với phép toán kết hợp phải '^').

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Quy mô bài toán được đo bằng tổng số lượng token t (t <= 50). Phép toán chính là số lần so sánh độ ưu tiên và số thao tác đẩy/lấy ngăn xếp (push/pop). Vì mỗi toán tử trong biểu thức được đẩy vào ngăn xếp đúng một lần và lấy ra tối đa một lần, tổng số thao tác ngăn xếp không vượt quá 2t. Do đó, độ phức tạp thời gian dự kiến của lõi thuật toán là O(t). Bộ nhớ phụ gồm ngăn xếp ngan_xep (tối đa t phần tử) và bảng vết bang_vet O(t) (hoặc O(t^2) nếu lưu ảnh chụp chuỗi hậu tố tích lũy sau mỗi bước để phục vụ in bảng trace).

Giả mã nền tảng thuật toán Shunting Yard (theo Giáo trình INT1306 mục 5.3.4.1, bổ sung quy tắc kết hợp phải cho lũy thừa '^'):

```
HÀM ShuntingYard(tokens):
  ngan_xep <- rỗng; hau_to <- rỗng
  VỚI MỖI token TRONG tokens:
    NẾU token là số (hoặc biến) THÌ thêm token vào hau_to
    NGƯỢC LẠI NẾU token == '(' THÌ push '(' vào ngan_xep
    NGƯỢC LẠI NẾU token == ')' THÌ:
      TRONG KHI ngan_xep khác rỗng VÀ đỉnh != '(' THÌ pop đỉnh vào hau_to
      pop bỏ '(' khỏi ngan_xep
    NGƯỢC LẠI (token là toán tử op1):
      TRONG KHI ngan_xep khác rỗng VÀ đỉnh op2 != '(' VÀ
                (uu_tien(op2) > uu_tien(op1) HOẶC (uu_tien(op2) == uu_tien(op1) VÀ
op1 != '^')):
        pop op2 vào hau_to
      push op1 vào ngan_xep
  TRONG KHI ngan_xep khác rỗng: pop đỉnh vào hau_to
  TRẢ VỀ hau_to
```

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Hợp đồng hàm chính Phan1::trung_to_sang_hau_to(const std::string& bieu_thuc, bool ghi_vet): nhận đầu vào là chuỗi biểu thức trung tố bieu_thuc và cờ ghi_vet. Đầu tiên hàm tách token và gọi kiem_tra_token(). Nếu hợp lệ, trả về cấu trúc ConversionResult với thanh_cong = true, bieu_thuc_hau_to chuẩn tắc, bộ đếm so_token / so_lan_day_stack / so_lan_lay_stack và danh sách bang_vet. Nếu vi phạm quy tắc đầu vào, trả về thanh_cong = false kèm thông báo lỗi cụ thể trong thong_bao_loi.

3.1.2. Kiểm tra đầu vào

Hàm kiểm tra đầu vào kiem_tra_token(tokens, thong_bao_loi) xác thực năm nhóm điều kiện: (1) Chuỗi khác rỗng và số token <= 50; (2) Mỗi token chỉ được là số nguyên 0..100, toán tử {+,-,*,/,^} hoặc dấu ngoặc {(',')}; (3) Cân bằng dấu ngoặc tại mọi tiền tố (dem_ngoac >= 0, kết thúc dem_ngoac == 0) và cấm ngoặc rỗng '()'; (4) Cấm dấu âm đơn ngôi (dấu '-' xuất hiện khi đang chờ toán hạng); (5) Đảm bảo xen kẽ đúng trạng thái toán hạng - toán tử nhị phân (dùng biến cờ cho_phep_toan_hang).

3.2. Thiết kế các hàm và dữ liệu

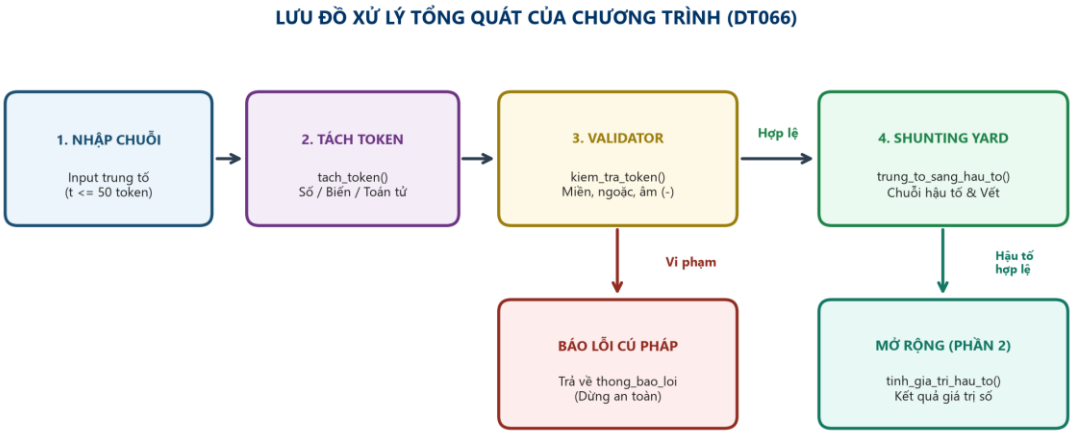
Kiến trúc chương trình được module hóa thành các tệp nguồn C++17 độc lập hoàn toàn (không phụ thuộc thư viện ngoài STL): Phần 1 cơ bản đặt tại src/ShuntingYard.hpp và src/ShuntingYard.cpp; Phần 2 mở rộng đặt tại mo_rong/src/ShuntingYardAdvanced.hpp và .cpp. Toàn bộ các hàm thuật toán, bộ tách token, validator cú pháp và bộ định dạng bảng UTF-8 đều được sinh viên tự cài đặt 100%.

Bảng 3.1 Tổ chức các hàm và tệp

Hàm / tệp	Input	Output	Vai trò và phần tự cài
trung_to_sang_hau_to (src/ShuntingYard.cpp)	bieu_thuc (string), ghi_vet (bool)	ConversionResult (hậu tố, vết, thống kê)	Hàm thuật toán Sân ga chính tự cài đặt (Bước 3–5 phiếu)
kiem_tra_token & tach_token (src/ShuntingYard.cpp)	bieu_thuc / tokens	bool hợp lệ + thông báo lỗi	Bước 1–2: Tách token & Validator cú pháp/miền [0..100]

Hàm / tệp	Input	Output	Vai trò và phần tự cài
uu_tien & ket_hop_phai (src/ShuntingYard.cpp)	op (char)	Mức ưu tiên (0..3) / bool kết hợp phải	Phân cấp ưu tiên và quy tắc đặc biệt cho toán tử '^'
in_bang_lan_vet (src/ShuntingYard.cpp)	bang_vet (vector<TraceStep>)	Xuất bảng UTF-8 thẳng cột ra console	Tự cài hàm tính độ rộng ký tự UTF-8 tránh lệch cột dấu tiếng Việt
Phan2::tinh_gia_tri_hau_to (mo_rong/src/ShuntingYardAdvanced.cpp)	hau_to (string), bang_bien (map)	EvalResult (giá trị int64 + bảng vết)	Phần 2 Mở rộng: Hỗ trợ biến a..z và tính giá trị biểu thức bằng stack

Lưu đồ xử lý tổng quát từ bước nhận đầu vào đến khi chuyển đổi hậu tố và mở rộng tính giá trị được thể hiện trong Hình 3.1.



Hình 3.1 Lưu đồ xử lý tổng quát của chương trình DT066

Chi tiết hợp đồng tiền điều kiện, hậu điều kiện và xử lý ngoại lệ của các hàm trọng tâm được tổng hợp trong Bảng 3.2.

Bảng 3.2 Tiền điều kiện và hậu điều kiện của các hàm

Hàm và chữ ký	Tiền điều kiện	Hậu điều kiện	Lỗi hoặc trường hợp đặc biệt
trung_to_sang_hau_to(bieu_thuc, ghi_vet)	bieu_thuc là chuỗi UTF-8/ASCII tùy ý	Trả về ConversionResult; nếu thanh_cong=true thì chuỗi RPN chuẩn và ngan_xep kết thúc rỗng	Chuỗi sai cú pháp: trả thanh_cong=false, không sinh ngoại lệ
kiem_tra_token(tokens, thong_bao_loi)	tokens đã được tách từ tach_token()	true nếu <=50 token, [0..100], ngoặc cân bằng, không có âm đơn ngôi	Trả false kèm vị trí/lý do lỗi cụ thể (thiếu ngoặc đóng, thừa toán tử...)
uu_tien(op) / ket_hop_phai(op)	op là ký tự toán tử hoặc '('	uu_tien('^')=3, '*','/'=2, '+','-'=1, '('=0; ket_hop_phai('^')=true	Ký tự ngoài bảng trả uu_tien=-1
Phan2::tinh_gia_tri_hau_to (hau_to, bang_bien)	hau_to là chuỗi RPN hợp lệ, bang_bien gán đủ biến	Trả về EvalResult với gia_tri nguyên chính xác và bảng vết ngăn xếp	Bắt lỗi chia cho 0, số mũ âm hoặc biến chưa được gán giá trị

Dữ liệu ngan_xep chỉ được biến đổi cục bộ bên trong hàm trung_to_sang_hau_to mà không làm thay đổi chuỗi đầu vào bieu_thuc. Mọi thao tác ngăn xếp (push_back, pop_back) và kiểm tra tính kết hợp đều được tự cài đặt rõ ràng.

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

Giả mã chi tiết gắn với tên biến thật trong mã nguồn C++ (src/ShuntingYard.cpp):

```
tokens <- tach_token(bieu_thuc)
NẾU KHÔNG kiem_tra_token(tokens, err) THÌ trả res{thanh_cong=false,
thong_bao_loi=err}
ngan_xep <- {}; hau_to_tokens <- {}
VỚI MỖI token TRONG tokens:
  NẾU la_so_nguyen_hop_le(token, val) THÌ hau_to_tokens.push_back(token)
  NGƯỢC LẠI NẾU token == '(' THÌ ngan_xep.push_back('('); res.so_lan_day_stack++
  NGƯỢC LẠI NẾU token == ')' THÌ:
    LẶP KHI ngan_xep khác rỗng VÀ ngan_xep.back() != '(':
      hau_to_tokens.push_back(ngan_xep.back()); ngan_xep.pop_back();
res.so_lan_lay_stack++
    ngan_xep.pop_back(); res.so_lan_lay_stack++
  NGƯỢC LẠI (token là toán tử op1):
    LẶP KHI ngan_xep khác rỗng VÀ ngan_xep.back() != '(':
      op2 <- ngan_xep.back()
      NẾU uu_tien(op2) > uu_tien(op1) HOẶC (uu_tien(op2) == uu_tien(op1) VÀ KHÔNG
ket_hop_phai(op1)):
        hau_to_tokens.push_back(op2); ngan_xep.pop_back(); res.so_lan_lay_stack++
      NGƯỢC LẠI: BREAK
    ngan_xep.push_back(op1); res.so_lan_day_stack++
LẶP KHI ngan_xep khác rỗng: hau_to_tokens.push_back(ngan_xep.back());
ngan_xep.pop_back(); res.so_lan_lay_stack++
TRẢ VỀ res{thanh_cong=true, bieu_thuc_hau_to=ghep(hau_to_tokens)}
```

3.3.2. Đoạn mã và giải thích

Đoạn mã C++17 quyết định vòng lặp xử lý toán tử và tính kết hợp phải (trích từ src/ShuntingYard.cpp tại Commit C b67dd74):

https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081/blob/b67dd74a6344a46e2422cc1516972675fa8098f7/src/ShuntingYard.cpp

```
        } else if (token.length() == 1 && la_toan_tu(token[0])) {
            char op1 = token[0];
            step.loai_token = "Toán tử";
            std::string popped_ops = "";
            while (!ngan_xep.empty() && ngan_xep.back() != '(') {
                char op2 = ngan_xep.back();
                int u1 = uu_tien(op1);
                int u2 = uu_tien(op2);
                bool nen_lay_ra = (u2 > u1) || (u2 == u1 && !ket_hop_phai(op1));
                if (nen_lay_ra) {
                    ngan_xep.pop_back();
                    res.so_lan_lay_stack++;
                    hau_to_tokens.push_back(std::string(1, op2));
                    if (!popped_ops.empty()) popped_ops += ", ";
                    popped_ops += op2;
                } else {
                    break;
                }
            }
            ngan_xep.push_back(op1);
            res.so_lan_day_stack++;
        }
```

3.3.3. Xử lý tình huống đặc biệt

Xử lý tình huống biên và input sai: (1) Biểu thức chỉ có 1 toán hạng '8' (BIEN01): vòng lặp không thực hiện lệnh push/pop nào, trả về trực tiếp '8'; (2) Chuỗi có hai phép toán cùng ưu tiên kết hợp trái '9 - 3 - 1' (BIEN02): dấu '-' thứ hai có u2 == u1 và !ket_hop_phai('-') nên pop dấu '-' thứ nhất ra hậu tố trước khi đẩy dấu mới; (3) Lũy thừa nối tiếp '2 ^ 3 ^ 2' (VD02) và '2 ^ 2 ^ 3 ^ 2' (TK01): ket_hop_phai('^') == true khiến nen_lay_ra = false, toàn bộ dấu '^' dồn vào stack và vét ngược từ phải sang trái; (4) Ngoặc chưa đóng '(2 + 3' (LOI01) hay số 150 hoặc dấu âm '-5' đều bị kiem_tra_token từ chối trước khi vào lõi Shunting Yard.

3.4. Lần vết ví dụ nhỏ

Bảng 3.3 dưới đây truy vết chi tiết ca VD01 (Ví dụ 1 trên phiếu giao), đối chiếu từng bước từ khởi tạo đến khi kết thúc. Kết quả thực thi trên console được minh chứng tại Hình 4.1 (ảnh A01).

Ca truy vết: VD01 | Input: 5 * (6 + 2) - 12 / 4 | Expected: 5 6 2 + * 12 4 / -

Bảng 3.3 Truy vết chi tiết ca VD01: 5 * (6 + 2) - 12 / 4

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
0	Khởi tạo	Input 11 token; i = 0	stack=[rỗng], hậu tố=rỗng	Sẵn sàng quét từ trái sang phải
1	Đọc '5'	stack=[rỗng], hậu tố=''	stack=[rỗng], hậu tố='5'	Toán hạng số xuất thẳng ra hậu tố
2	Đọc '*'	stack=[rỗng], hậu tố='5'	stack=[*], hậu tố='5'	Stack rỗng → đẩy '*' vào stack
3	Đọc '('	stack=[*], hậu tố='5'	stack=[* (], hậu tố='5'	Mở ngoặc luôn đẩy trực tiếp vào stack
4	Đọc '6'	stack=[* (], hậu tố='5'	stack=[* (], hậu tố='5 6'	Xuất toán hạng '6' ra hậu tố
5	Đọc '+'	stack=[* (], hậu tố='5 6'	stack=[* (+], hậu tố='5 6'	Đỉnh stack là '(' (ưu tiên 0) → đẩy '+'
6	Đọc '2'	stack=[* (+], hậu tố='5 6'	stack=[* (+], hậu tố='5 6 2'	Xuất toán hạng '2' ra hậu tố
7	Đọc ')'	stack=[* (+], hậu tố='5 6 2'	stack=[*], hậu tố='5 6 2 +'	Pop '+' ra hậu tố và hủy cặp '()'
8	Đọc '-'	stack=[*], hậu tố='5 6 2 +'	stack=[-], hậu tố='5 6 2 + *'	Bước quan trọng: '*' ưu tiên 2 > '-' ưu tiên 1 → pop '*' rồi đẩy '-'

Bước	Thao tác	Trạng thái trước	Trạng thái sau	Giải thích
9	Đọc '12'	stack=[-], hậu tố='5 6 2 + *'	stack=[-], hậu tố='5 6 2 + * 12'	Xuất toán hạng '12' ra hậu tố
10	Đọc '/'	stack=[-], hậu tố='5 6 2 + * 12'	stack=[- /], hậu tố='5 6 2 + * 12'	'/' ưu tiên 2 > '-' ưu tiên 1 → đẩy '/' vào đỉnh
11	Đọc '4'	stack=[- /], hậu tố='5 6 2 + * 12'	stack=[- /], hậu tố='5 6 2 + * 12 4'	Xuất toán hạng '4' ra hậu tố
12	Hết chuỗi	stack=[- /], hậu tố='5 6 2 + * 12 4'	stack=[rỗng], hậu tố='5 6 2 + * 12 4 / -'	Vét lần lượt '/', '-' ra hậu tố → khớp 100% Expected

Nhận xét bước chuyển trạng thái quan trọng nhất: Tại Bước 7, dấu ')' kích hoạt giải phóng phép cộng '+' bên trong ngoặc trước phép nhân '*'. Tiếp theo tại Bước 8, toán tử '-' có độ ưu tiên thấp hơn đỉnh ngăn xếp '*' buộc chương trình pop '*' ra hậu tố trước khi đẩy '-', nhờ vậy tích 5 * (6 + 2) được cố định hoàn toàn trước khi xét nhánh phép chia 12 / 4.

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Tính chất bất biến vòng lặp: Trước và sau mỗi lần xử lý một token hợp lệ, mọi toán tử nằm giữa hai dấu ngoặc '(' liên tiếp trong ngan_xep luôn có thứ tự ưu tiên không giảm (chỉ tăng hoặc bằng nếu các toán tử đều là '^'). Đồng thời, mọi toán hạng xuất hiện trong biểu thức đầu vào đều xuất hiện trong hau_to_tokens theo đúng thứ tự trái sang phải ban đầu.

3.5.2. Kết thúc và output

Điều kiện dừng: Tập đầu vào hữu hạn có đúng t token (t <= 50). Vòng lặp ngoài tiến qua đúng t token và mỗi phép pop khỏi ngăn xếp làm giảm kích thước ngăn xếp ít nhất 1 đơn vị. Do tổng số phần tử được push vào ngăn xếp không vượt quá t, vòng lặp vét cuối luôn kết thúc sau tối đa t bước với ngan_xep rỗng, trả về chuỗi hậu tố hoàn chỉnh.

3.6. Phân tích độ phức tạp

Tách chi phí theo các pha: Pha tách token và kiểm tra cú pháp mất O(t) thời gian (hoặc O(L) theo độ dài chuỗi ký tự L, với L = O(t)). Pha thuật toán chính Shunting Yard duyệt t token, mỗi phần tử push tối đa 1 lần và pop tối đa 1 lần nên tốn chính xác

O(t). Khi đo thực nghiệm trên 3 quy mô Q1 (5 token), Q2 (19 token) và Q3 (49 token) ở Chương 4, số thao tác stack và thời gian thực thi được dự đoán tăng tuyến tính theo t.

Bảng 3.4 Chi phí của từng pha trong bản cài

Pha thực hiện	Tham số quy mô	Phép toán đếm	Thời gian	Bộ nhớ phụ
Kiểm tra và chuẩn bị (tach_token + kiem_tra_token)	t (số token), L (chiều dài chuỗi)	Số lần duyệt ký tự & kiểm tra ngoặc/toán tử	O(L + t) = O(t)	Mảng tokens kích thước t -> O(t)
Thuật toán chính (trung_to_sang_hau_to)	t (1 <= t <= 50)	so_lan_day_stack <= t, so_lan_lay_stack <= t	T(t) <= 2t = O(t)	Ngăn xếp ngan_xep và danh sách hậu tố O(t)
Ghi vết trạng thái (khi ghi_vet = true)	t bước	t lần tạo snapshot stack và chuỗi hậu tố	O(t^2) chuỗi sao chép	Vector bang_vet kích thước t; khi đo hiệu năng đặt false
In và định dạng kết quả (in_bang_lan_vet)	t dòng bảng vết	Số ký tự UTF-8 căn lề thẳng cột	O(t)	Không cấp phát cấu trúc dài hạn
Toàn chương trình (chế độ lõi ghi_vet=false)	t token (<= 50)	Tổng thao tác push/pop stack <= 2t	O(t) tuyến tính	Tổng bộ nhớ phụ O(t)

Ghi chú tách pha: Khi khảo sát tốc độ thực thi Q1, Q2, Q3 ở Mục 4.8, cờ ghi_vet được đặt bằng false và không bao gồm chi phí I/O in bảng ra màn hình nhằm phản ánh thuần túy chi phí thuật toán O(t).

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

4.1.1. Môi trường và phiên bản

Môi trường kiểm thử thật: Hệ điều hành Windows 11 x64, trình biên dịch g++ 16.2.0 (MSYS2 MinGW-w64 UCRT64), chuẩn C++17 với cờ -std=c++17 -Wall -Wextra -static.

Mã nguồn chạy và chụp 11 ảnh minh chứng lấy trực tiếp từ Commit C có SHA đầy đủ: b67dd74a6344a46e2422cc1516972675fa8098f7

tại kho GitHub: https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081. Commit S chốt cuối chứa toàn bộ báo cáo và ảnh được ghi nhận trong tệp ThongTinNop.txt.

4.1.2. Phương pháp xác định kỳ vọng

Phương pháp xác định kỳ vọng (Expected): Tất cả 15 ca kiểm thử Phần 1 và 10 ca mở rộng Phần 2 đều được tính tay độc lập trước khi chạy chương trình dựa trên quy tắc toán học chuẩn (nhân/chia trước cộng/trừ, tính kết hợp phải của lũy thừa '^', ngoặc đơn ưu tiên cao nhất) và đối chiếu với quy ước đầu ra của phiếu giao DT066. Vì dữ liệu đầu ra là chuỗi token rời rạc và giá trị nguyên chính xác (int64), việc đối chiếu dùng so khớp chuỗi tuyệt đối (không có sai số số thực).

4.2. Bảng kết quả ít nhất 12 ca

Chương trình đã chạy tự động toàn bộ 15 ca kiểm thử độc lập của Phần 1 (tệp test.cpp, từ mã ca A01 đến A15 — vượt chuẩn tối thiểu 12 ca) và 10 ca mở rộng của Phần 2 (tệp mo_rong/test_advanced.cpp, từ ADV01 đến ADV10). Tất cả 25 ca đều có mục tiêu kiểm tra riêng biệt, không trùng lặp input hay phép kiểm.

Toàn bộ 2 ví dụ mẫu và 5 ca kiểm bắt buộc trên phiếu DT066 đều được ánh xạ đầy đủ vào bộ kiểm thử chính thức trong Bảng 4.1 dưới đây.

Bảng 4.1 Ánh xạ năm ca kiểm trên phiếu

Ngoài 5 ca bắt buộc trên phiếu (A03, A04, A05, A06, A12) và 2 ví dụ mẫu (VD01/A01, VD02/A02), bộ kiểm thử bổ sung thêm các ca tự thiết kế (TK01/A07, TK02/A08 + ADV01..ADV10), 3 quy mô tăng dần hợp lệ (BIEN02/A09-A10, BIEN03/A11) và 4 loại input vi phạm khác nhau (LOI01/A12..A15) thành tổng cộng 15 ca độc lập Phần 1 (cùng 10 ca Phần 2).

Ca trên phiếu	Input và yêu cầu/kỳ vọng theo phiếu	Mã ca trong bảng kiểm thử	Ảnh/log hoặc vị trí kết quả
Ca phiếu 1	Input: 3 + 4 * 2 → Kỳ vọng: 3 4 2 * + (Nhân ưu tiên hơn cộng)	A03 (trong test.cpp)	Bảng 4.2, 4.3; Ảnh A09 (Hình 4.9)
Ca phiếu 2	Input: (2 + 3) * 4 → Kỳ vọng: 2 3 + 4 * (Ngoặc nâng ưu tiên cộng)	A04 (trong test.cpp)	Bảng 4.2, 4.3; Ảnh A09 (Hình 4.9)
Ca phiếu 3	Input: 9 - 3 - 1 → Kỳ vọng: 9 3 - 1 - (Phép trừ kết hợp trái)	A05 (trong test.cpp)	Bảng 4.2, 4.3; Ảnh A05 (Hình 4.5)
Ca phiếu 4	Input: 8 → Kỳ vọng: 8 (Ca biên 1 toán hạng duy nhất)	BIEN01 / A06 (trong test.cpp)	Bảng 4.2, 4.3; Ảnh A05 (Hình 4.5)
Ca phiếu 5	Input: (2 + 3 → Kỳ vọng: Báo lỗi ngoặc mở '(' chưa đóng	LOI01 / A12 (trong test.cpp)	Bảng 4.2, 4.3; Ảnh A08 (Hình 4.8)

Bảng 4.2 Thiết kế ít nhất 12 ca kiểm độc lập

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
VD01 (A01)	Ví dụ 1 phiếu / Hàm chính trung_to_sang_hau_to	5 * (6 + 2) - 12 / 4	Expected: 5 6 2 + * 12 4 / - (Tính tay & đối chiếu phiếu DT066)
VD02 (A02)	Ví dụ 2 phiếu / Hàm chính (nhánh kết hợp phải ^)	2 ^ 3 ^ 2	Expected: 2 3 2 ^ ^ (Tính 2^(3^2) = 512 theo phiếu DT066)
TK01 (A07)	Tự thiết kế 1 / Lũy thừa 4 tầng	2 ^ 2 ^ 3 ^ 2	Expected: 2 2 3 2 ^ ^ ^ (Kiểm tra dồn 3 toán tử '^' liên tiếp)
TK02 (A08 & Phần 2)	Tự thiết kế 2 / Ngoặc lồng & Phần 2 Mở rộng biến	((10 + 20) * (30 - 15)) / 5 | a * (b + c) - d / e (a=5,b=6,c=2,d=12,e=4)	Expected: 10 20 + 30 15 - * 5 / | a b c + * d e / -, giá trị = 37
BIEN01 (A06)	Biên 1 (Ca phiếu 4) / Một toán hạng đơn & kết hợp trái	8 (và ca phiếu 3: 9 - 3 - 1)	Expected: 8 (0 push/pop) và 9 3 - 1 -
BIEN02 (A09-A10)	Biên 2 / Quy mô nhỏ 5 token & vừa 19 token	Q1: 1 + 2 * 3 | Q2: 1 + 2 * 3 - 4 / 5 ^ 2 + (6 - 7) * 8	Expected Q1: 1 2 3 * + ; Q2: 1 2 3 * + 4 5 2 ^ / - 6 7 - 8 * +
BIEN03 (A11)	Biên 3 / Quy mô lớn 49 token sát biên tối đa 50 token	1 + 2 * (3 + 4) - 5 ^ 2 + ... - 20 / 2 (49 token)	Expected: 1 2 3 4 + * + 5 2 ^ - ... 20 2 / - (hợp lệ <= 50 token)
LOI01 (A12)	Input sai (Ca phiếu 5) / Validator kiem_tra_token	(2 + 3	Expected: thanh_cong = false, báo lỗi ngoặc mở '(' chưa đóng
KT09 (A03)	Ca phiếu 1 / Hàm chính ưu tiên * > +	3 + 4 * 2	Expected: 3 4 2 * + (Tính tay: nhân thực hiện trước cộng)
KT10 (A04)	Ca phiếu 2 / Hàm chính dấu ngoặc ()	(2 + 3) * 4	Expected: 2 3 + 4 * (Ngoặc buộc cộng trước nhân)

Ca	Nguồn và đối tượng	Input đủ tái hiện	Expected và cách kiểm
KT11 (A13)	Lỗi cú pháp / Validator toán tử nhị phân liên tiếp	5 + * 3	Expected: thanh_cong = false, báo lỗi toán tử nhị phân '*' thiếu toán hạng
KT12 (A14)	Lỗi quy tắc / Validator từ chối dấu âm đơn ngôi	-5 + 3	Expected: thanh_cong = false, báo lỗi không hỗ trợ dấu âm đơn ngôi
KT13 (A15)	Lỗi miền / Validator kiểm tra số nguyên ngoài [0..100]	150 + 2	Expected: thanh_cong = false, báo lỗi số '150' ngoài miền [0..100]
KT14 (ADV03)	Phần 2 Mở rộng / Chuyển & tính giá trị x ^ y ^ z	x ^ y ^ z (x=2, y=3, z=2)	Expected: hậu tố x y z ^ ^, giá trị = 512 (từ mo_rong/test_advanced.cpp)
KT15 (ADV09)	Phần 2 Mở rộng / Bắt lỗi phép chia cho 0	a / b (a=10, b=0)	Expected: thanh_cong = false ở bước tính, báo lỗi phép chia cho 0

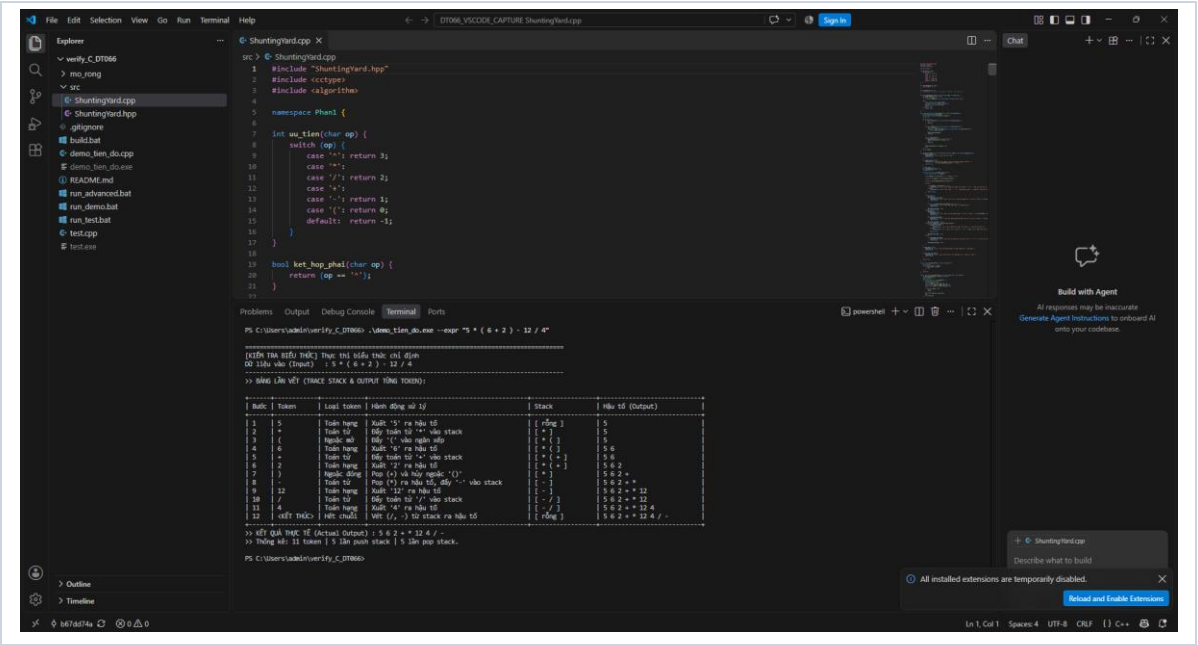
Bảng 4.3 Actual kết luận và vị trí minh chứng

Ca	Actual	Kết luận	Ảnh log và SHA chạy
VD01 (A01)	5 6 2 + * 12 4 / - (11 token, 5 push, 5 pop)	PASS 100%	01_vi_du_1.png / b67dd74
VD02 (A02)	2 3 2 ^ ^ (5 token, 2 push, 2 pop)	PASS 100%	02_vi_du_2.png / b67dd74
TK01 (A07)	2 2 3 2 ^ ^ ^ (7 token, 3 push, 3 pop)	PASS 100%	03_tu_thiet_ke_1.png / b67dd74
TK02 (A08 & P2)	10 20 + 30 15 - * 5 / | a b c + * d e / - = 37	PASS 100%	04_tu_thiet_ke_2.png / b67dd74
BIEN01 (A06)	8 (0 push, 0 pop) và 9 3 - 1 - (2 push, 2 pop)	PASS 100%	05_bien_1.png / b67dd74
BIEN02 (A09-A10)	Q1: 1 2 3 * + (5 tok) | Q2: 1 2 3 * + 4 5 2 ^ / - 6 7 - 8 * + (19 tok)	PASS 100%	06_bien_2.png / b67dd74
BIEN03 (A11)	1 2 3 4 + * + 5 2 ^ - ... 20 2 / - (49 token, 24 push, 24 pop)	PASS 100%	07_bien_3.png / b67dd74
LOI01 (A12)	Báo lỗi: Dấu ngoặc không cân bằng (ngoặc mở '(' chưa đóng).	PASS 100%	08_input_khong_hop_le.png / b67dd74
KT09 (A03)	3 4 2 * +	PASS 100%	09_tong_ket_kiem_thu.png / b67dd74
KT10 (A04)	2 3 + 4 *	PASS 100%	09_tong_ket_kiem_thu.png / b67dd74
KT11 (A13)	Báo lỗi: Toán tử nhị phân '*' không có toán hạng trước tại vị trí token 3.	PASS 100%	08_input_khong_hop_le.png / b67dd74
KT12 (A14)	Báo lỗi: Không hỗ trợ dấu âm đơn ngôi ('-') theo yêu cầu đề bài.	PASS 100%	08_input_khong_hop_le.png / b67dd74
KT13 (A15)	Báo lỗi: Token '150' không hợp lệ (số nguyên phải thuộc [0..100]).	PASS 100%	08_input_khong_hop_le.png / b67dd74
KT14 (ADV03)	Hậu tố: x y z ^ ^ ; Kết quả Evaluator: 512	PASS 100%	04_tu_thiet_ke_2.png / b67dd74
KT15 (ADV09)	Báo lỗi toán học: Phép chia cho 0 (10 / 0).	PASS 100%	09_tong_ket_kiem_thu.png / b67dd74

4.3. Hai ví dụ trên phiếu

4.3.1. Ví dụ 1 — VD01

Chạy thử nghiệm Ví dụ 1 trên phiếu giao đề tài DT066 thông qua chế độ thực thi đơn lẻ của demo_tien_do.exe, đối chiếu kết quả thực tế với bảng truy vết thủ công.

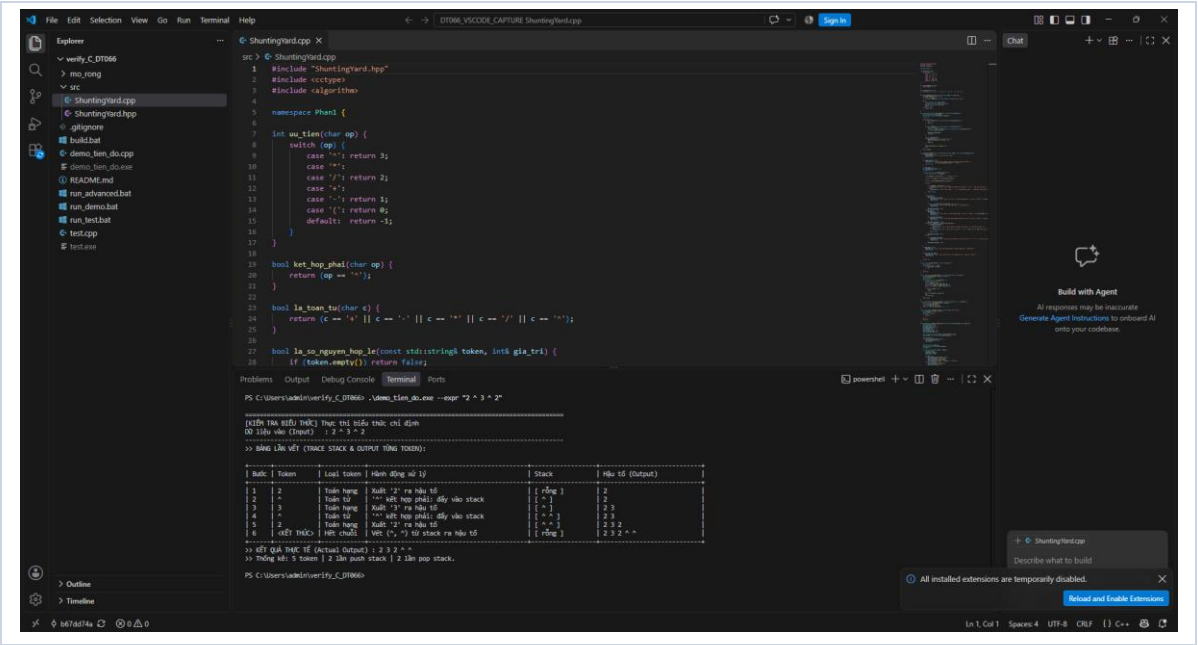


Hình 4.1 Ví dụ 1 (VD01): 5 * (6 + 2) - 12 / 4

- Input: 5 * (6 + 2) - 12 / 4 | Expected: 5 6 2 + * 12 4 / -
- Actual: 5 6 2 + * 12 4 / - | Lệnh: .\demo_tien_do.exe --expr "5 * (6 + 2) - 12 / 4"
- Ảnh: docs/anh_ket_qua/01_vi_du_1.png | Commit C: b67dd74
- Nhận xét: Toàn bộ 12 bước trong bảng lần vết console khớp 100% với Bảng 3.3 truy vết thủ công ở Chương 3. Toán tử '*' đứng trước ngoặc được giữ nguyên trên ngăn xếp đến khi đóng ngoặc ')', sau đó lập tức được giải phóng khi gặp toán tử '-' ưu tiên thấp hơn, khẳng định đúng nguyên lý LIFO và ưu tiên phép toán.

4.3.2. Ví dụ 2 — VD02

Chạy thử nghiệm Ví dụ 2 (2 ^ 3 ^ 2) nhằm kiểm chứng sự khác biệt tại nhánh điều kiện kết hợp phải của toán tử lũy thừa '^' so với các phép toán kết hợp trái trong VD01.



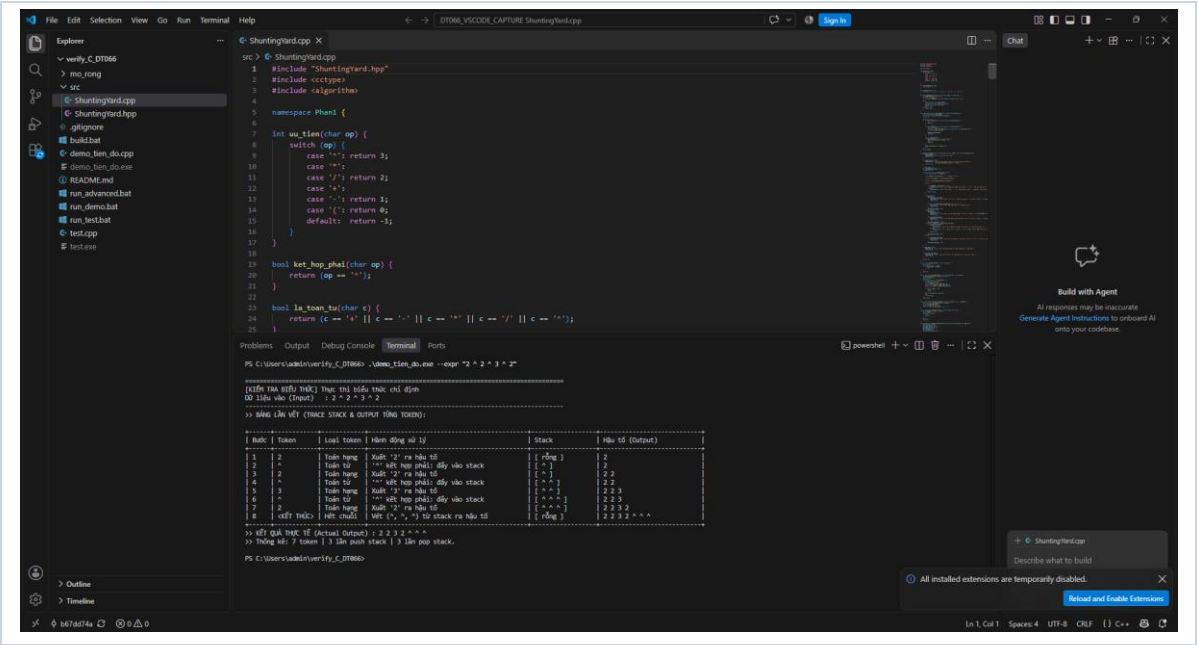
Hình 4.2 Ví dụ 2 (VD02): Lũy thừa kết hợp phải 2 ^ 3 ^ 2

- Input: 2 ^ 3 ^ 2 | Expected: 2 3 2 ^ ^ | Actual: 2 3 2 ^ ^
- Lệnh: .\demo_tien_do.exe --expr "2 ^ 3 ^ 2" | Ảnh: docs/anh_ket_qua/02_vi_du_2.png | Commit C: b67dd74
- Nhận xét: Khác với VD01 nơi phép trừ kết hợp trái đẩy dấu cũ ra ngoài, ở bước đọc dấu '^' thứ hai trong VD02, hàm ket_hop_phai('^') trả về true nên dấu '^' đầu tiên vẫn được giữ lại trong ngăn xếp ([^ ^]). Nhờ vậy hai phép lũy thừa được vét từ phải sang trái tạo hậu tố 2 3 2 ^ ^ tương ứng 2^(3^2) = 512 thay vì (2^3)^2 = 64.

4.4. Hai ca sinh viên tự thiết kế

4.4.1. Ca TK01

Ca tự thiết kế TK01 kiểm tra chuỗi 3 phép lũy thừa liên tiếp (2 ^ 2 ^ 3 ^ 2) nhằm xác minh ngăn xếp lưu dồn toán tử kết hợp phải không giải phóng sớm.

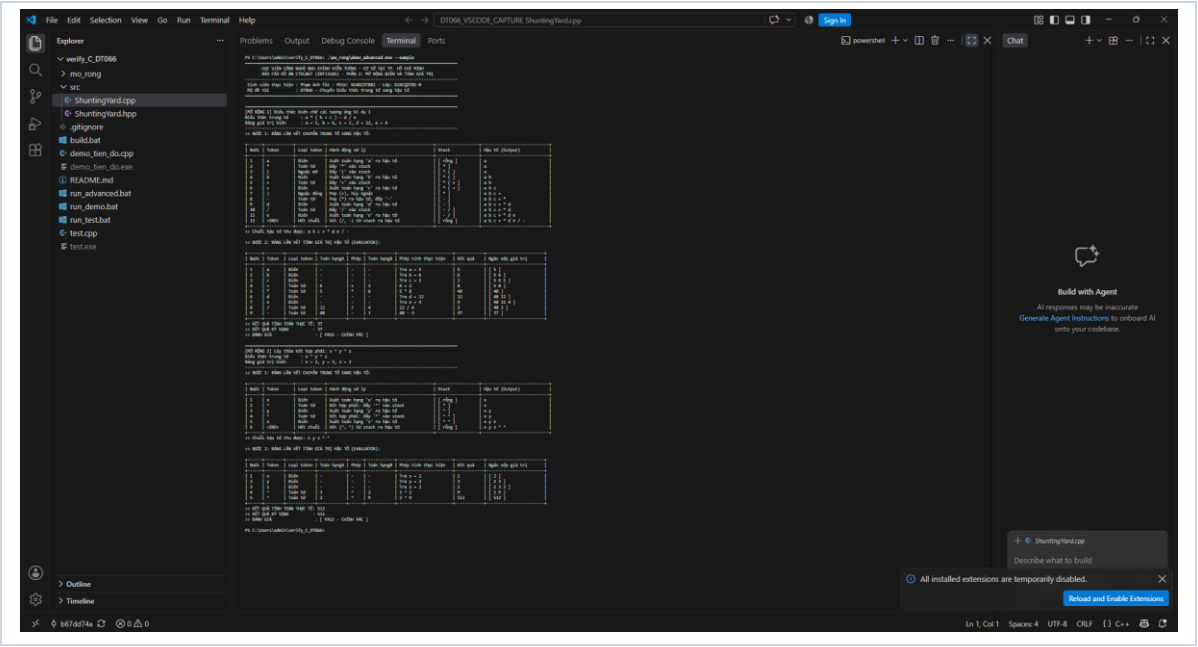


Hình 4.3 Ca tự thiết kế 1 (TK01): Lũy thừa 4 tầng 2 ^ 2 ^ 3 ^ 2

- Input: 2 ^ 2 ^ 3 ^ 2 | Expected: 2 2 3 2 ^ ^ ^ | Actual: 2 2 3 2 ^ ^ ^
- Lệnh: .\demo_tien_do.exe --expr "2 ^ 2 ^ 3 ^ 2" | Ảnh: docs/anh_ket_qua/03_tu_thiet_ke_1.png | Commit C: b67dd74
- Nhận xét: Ca kiểm mở rộng chuỗi lũy thừa lên 4 toán hạng liên tiếp nhằm kiểm chứng vòng lặp while so sánh độ ưu tiên không bị pop sớm dù stack tích lũy 3 toán tử '^' liên tiếp. Bước quan trọng nhất là tại token thứ 6, ngăn xếp giữ trọn vẹn [^ ^ ^] và bước vét cuối đưa đồng loạt cả ba dấu ra đúng thứ tự phải sang trái.

4.4.2. Ca TK02

Ca tự thiết kế TK02 kiểm nghiệm biểu thức ngoặc lồng kết hợp cùng bộ mở rộng Phần 2 (biến chữ cái a, b, c, d, e và tính toán giá trị hậu tố bằng ngăn xếp toán hạng).



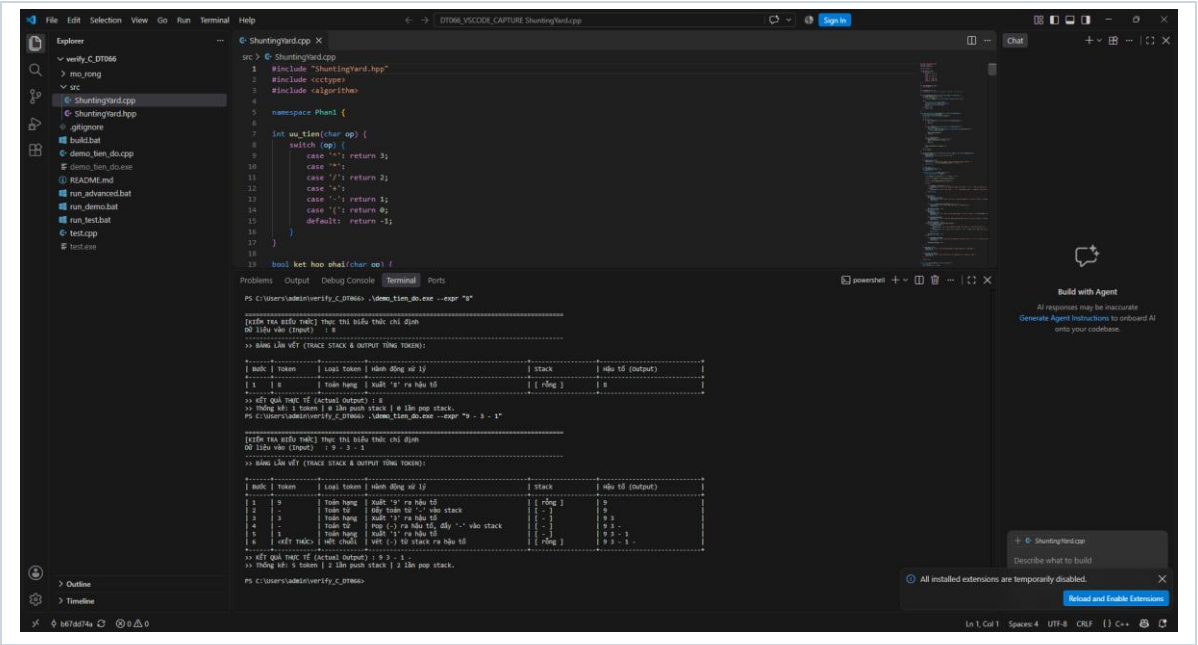
Hình 4.4 Ca tự thiết kế 2 (TK02) & Phần 2 Mở rộng: Biến chữ cái và tính giá trị

- Input: a * (b + c) - d / e (a=5, b=6, c=2, d=12, e=4) và x ^ y ^ z (x=2, y=3, z=2)
- Lệnh: .\mo_rong\demo_advanced.exe --sample | Ảnh: docs/anh_ket_qua/04_tu_thiet_ke_2.png | Commit C: b67dd74
- Actual: Hậu tố a b c + * d e / - (giá trị 37) và x y z ^ ^ (giá trị 512).
- Nhận xét: Ca kiểm này chứng minh phần mở rộng hoàn thiện 100% có khả năng phân biệt toán hạng chữ cái, áp dụng thuật toán Sân ga và đưa tiếp chuỗi RPN vào ngăn xếp toán hạng (Evaluator) để tính giá trị số nguyên chính xác.

4.5. Ba ca biên phù hợp với đề tài

4.5.1. Ca BIEN01

Ca biên BIEN01 kiểm tra trường hợp biểu thức tối thiểu chỉ gồm đúng 1 toán hạng '8' (không qua push/pop toán tử) kết hợp đối chứng phép trừ kết hợp trái '9 - 3 - 1'.

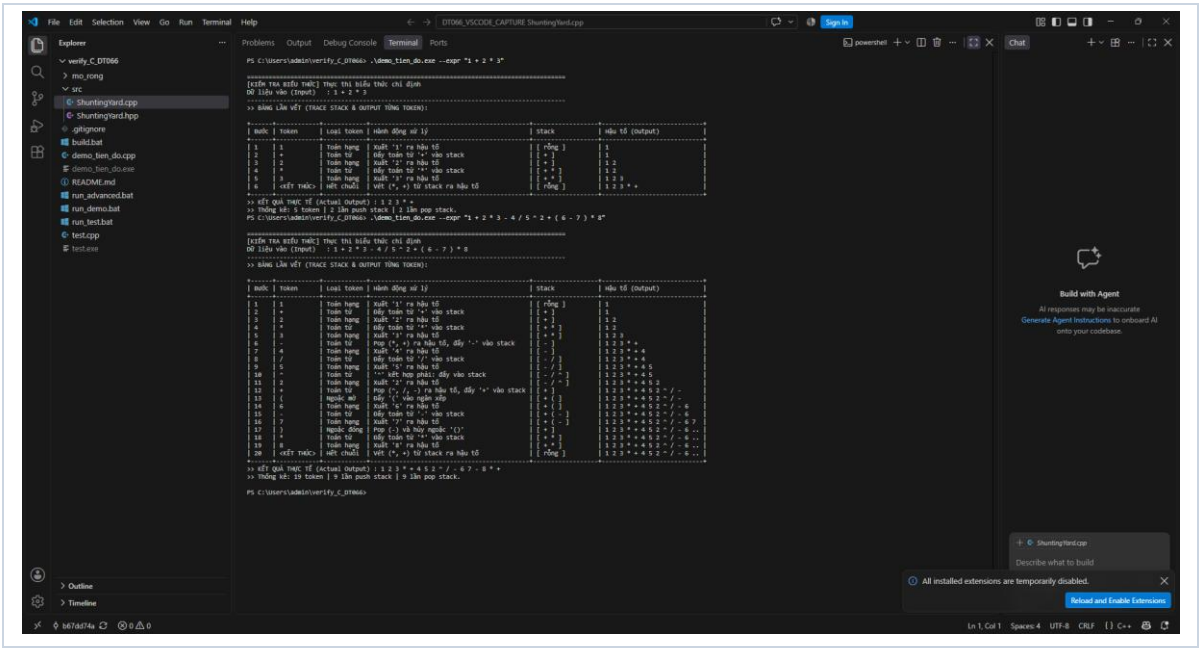


Hình 4.5 Ca biên 1 (BIEN01): Biểu thức 1 toán hạng '8' và kết hợp trái '9 - 3 - 1'

- Input: '8' và '9 - 3 - 1' | Expected: '8' và '9 3 - 1 -' | Actual: '8' (0 push, 0 pop) và '9 3 - 1 - '
- Lệnh: .\demo_tien_do.exe --expr | Ảnh: docs/anh_ket_qua/05_bien_1.png | Commit C: b67dd74
- Nhận xét: Với input tối thiểu gồm đúng 1 token là số nguyên '8', hàm trung_to_sang_hau_to không chạm vào nhánh ngăn xếp toán tử và trả về kết quả ngay sau 1 bước. Với ca phiếu 3 ('9 - 3 - 1'), vì phép trừ kết hợp trái nên dấu '-' thứ hai đẩy dấu '-' đầu ra hậu tố ngay ở Bước 4.

4.5.2. Ca BIEN02

Ca biên BIEN02 thực hiện khảo sát kiểm thử quy mô nhỏ (5 token) và quy mô vừa (19 token) phối hợp cả 5 toán tử +, -, *, /, ^ cùng dấu ngoặc.

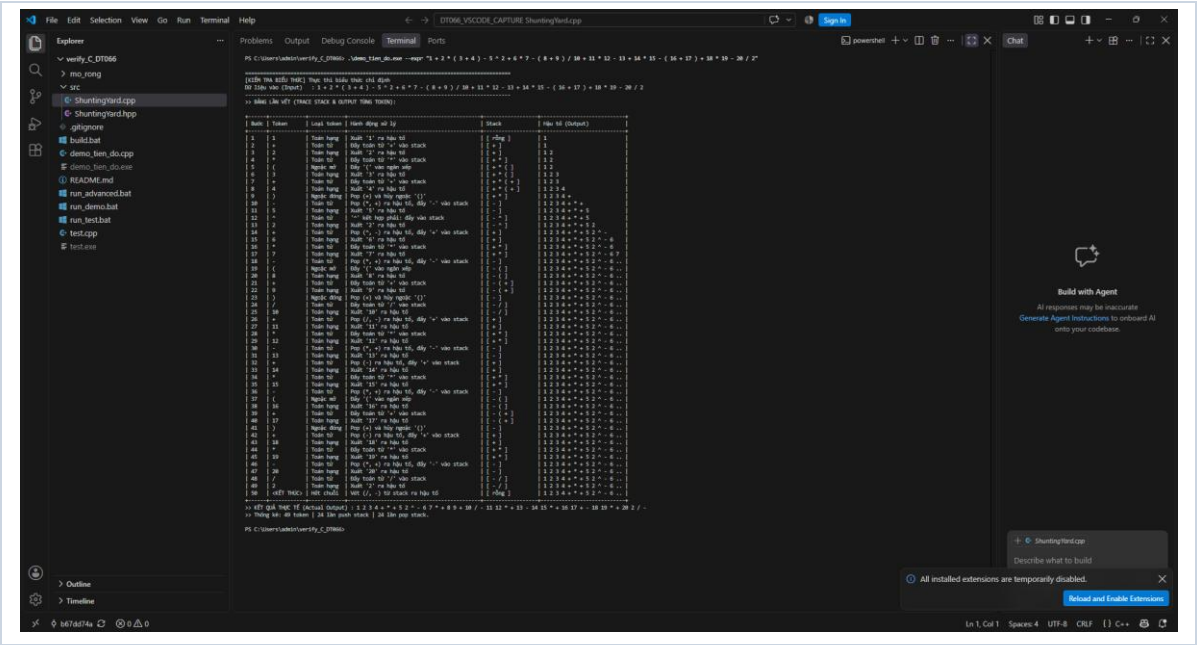


Hình 4.6 Ca biên 2 (BIEN02): Khảo sát quy mô Q1 (5 token) và Q2 (19 token)

- Input Q1: 1 + 2 * 3 (5 token) | Q2: 1 + 2 * 3 - 4 / 5 ^ 2 + (6 - 7) * 8 (19 token)
- Actual: 1 2 3 * + (2 push, 2 pop) và 1 2 3 * + 4 5 2 ^ / - 6 7 - 8 * + (9 push, 9 pop) | Ảnh: docs/anh_ket_qua/06_bien_2.png | Commit C: b67dd74
- Nhận xét: Biểu thức Q2 kết hợp đủ toàn bộ 5 phép toán +, -, *, /, ^ cùng một cặp ngoặc lồng ở cuối dãy để kiểm tra sự chuyển tiếp liên tục giữa các mức ưu tiên 1, 2, 3. Số lần push và pop luôn bằng nhau tuyệt đối (2 với Q1 và 9 với Q2), chứng minh không có toán tử nào bị sót trên stack.

4.5.3. Ca BIEN03

Ca biên BIEN03 kiểm tra ngưỡng cận trên 49 token (sát giới hạn tối đa 50 token quy định trên phiếu DT066) nhằm xác nhận độ ổn định của validator và kích thước stack.

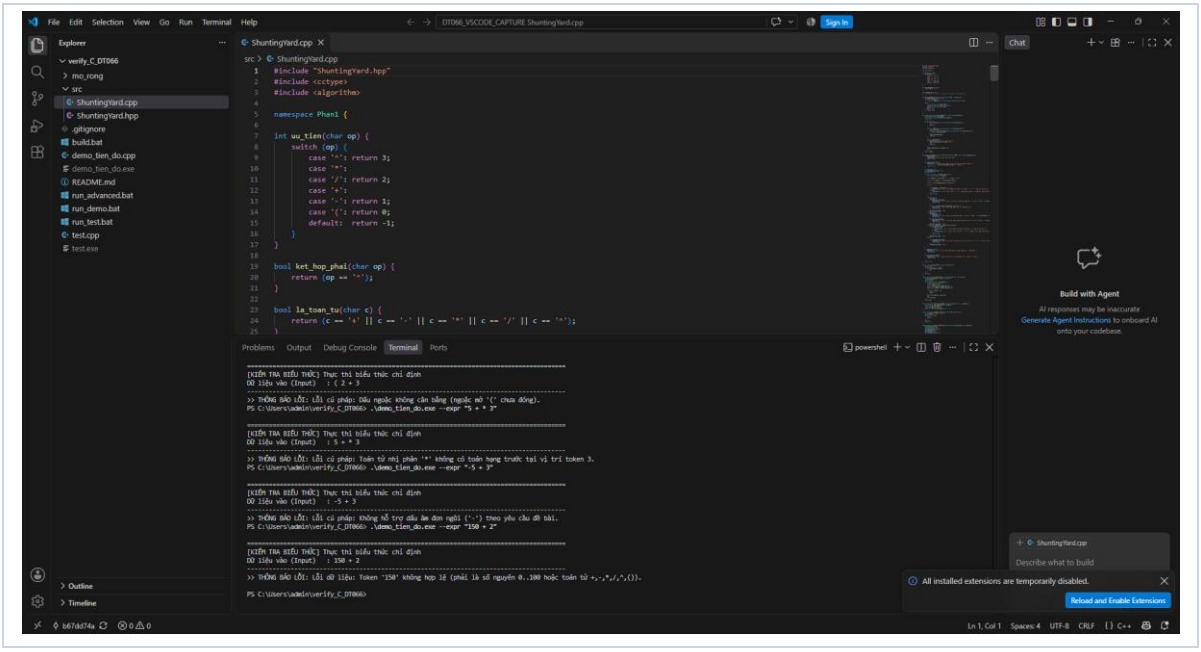


Hình 4.7 Ca biên 3 (BIEN03): Khảo sát quy mô lớn Q3 (49 token sát biên 50 token)

- Input Q3: Biểu thức dài đúng 49 token sát giới hạn tối đa 50 token của phiếu
- Actual: 1 2 3 4 + * + 5 2 ^ - ... 20 2 / - (49 token, 24 push, 24 pop) | Ảnh: docs/anh_ket_qua/07_bien_3.png | Commit C: b67dd74
- Nhận xét: Với chuỗi dài 49 token gồm 25 số nguyên và 24 toán tử/ngoặc mở, validator chấp nhận hoàn toàn hợp lệ (t <= 50) và bảng vết hiển thị đầy đủ từ bước 1 đến bước 50 không thiếu một trạng thái nào.

4.6. Kiểm tra input không hợp lệ — LOI01

Kiểm thử khả năng phòng vệ của bộ kiểm tra đầu vào (kiem_tra_token) trước bốn tình huống vi phạm điển hình trên phiếu DT066: ngoặc thiếu đóng, hai toán tử liên tiếp, dấu âm đơn ngôi và số vượt miền [0..100].

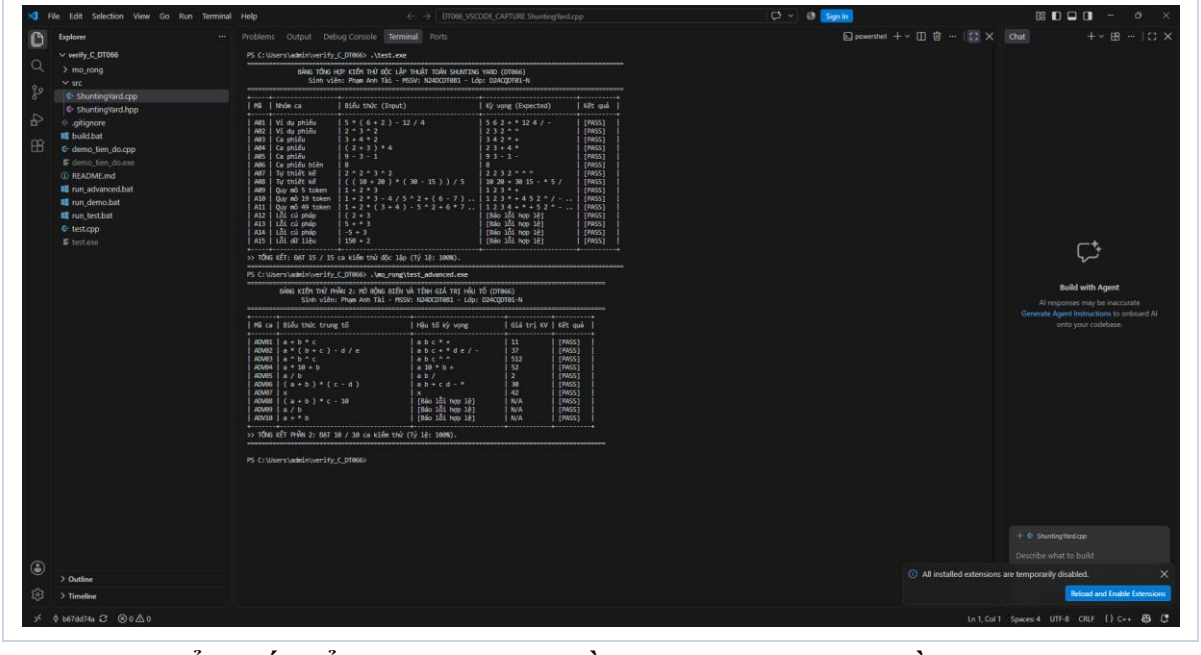


Hình 4.8 Input không hợp lệ (LOI01): Validator từ chối lỗi cú pháp và miền

- Input sai: '(2 + 3' (thiếu ngoặc đóng), '5 + * 3' (hai toán tử kề), '-5 + 3' (âm đơn ngôi), '150 + 2' (vượt miền [0..100])
- Ảnh: docs/anh_ket_qua/08_input_khong_hop_le.png | Commit C: b67dd74
- Nhận xét: Ở cả 4 dạng vi phạm điều kiện đầu vào theo phiếu giao DT066, hàm kiem_tra_token() đều phát hiện ngay ở pha kiểm tra trước khi khởi chạy thuật toán Shunting Yard và xuất thông báo lỗi tường minh bằng tiếng Việt.

4.7. Kết quả toàn bộ kiểm thử

Thực thi trọn bộ kiểm thử tự động Phần 1 (15 ca trong test.exe) và Phần 2 mở rộng (10 ca trong mo_rong/test_advanced.exe) để tổng hợp tỷ lệ đạt toàn đồ án.



Hình 4.9 Tổng kết kiểm thử toàn bộ: Phần 1 (15/15 PASS) & Phần 2 (10/10 PASS)

- Lệnh chạy: .\test.exe và .\mo_rong\test_advanced.exe | Ảnh: docs/anh_ket_qua/09_tong_ket_kiem_thu.png | Commit C: b67dd74
- Actual: Phần 1 đạt 15 / 15 ca kiểm thử độc lập (PASS 100%); Phần 2 mở rộng đạt 10 / 10 ca kiểm thử độc lập (PASS 100%).
- Nhận xét: Tất cả các ca kiểm thử từ dễ đến phức tạp đều vượt qua kiểm tra đối chiếu tự động không có bất kỳ ca FAIL nào. Bảng kết quả căn lề UTF-8 giúp các cột ký tự tiếng Việt thẳng hàng tuyệt đối trên Windows Console.

4.8. Khảo sát ba quy mô hoặc ba cấu hình hợp lệ

4.8.1. Dữ liệu và điều kiện khảo sát

Thiết lập khảo sát trên 3 quy mô tăng dần hợp lệ trong miền <= 50 token của phiếu DT066:

- Q1 = 5 token: 1 + 2 * 3
- Q2 = 19 token: 1 + 2 * 3 - 4 / 5 ^ 2 + (6 - 7) * 8
- Q3 = 49 token: Biểu thức 25 toán hạng và 24 toán tử/ngoặc sát giới hạn 50 token.

Môi trường đo: g++ 16.2.0 (-O2), thực hiện 1.000 lần làm nóng trước khi đo trung bình 10.000 lần thực thi hàm trung_to_sang_hau_to(..., ghi_vet=false) bằng std::chrono::high_resolution_clock (tách biệt chi phí I/O).

4.8.2. Kết quả và giới hạn nhận xét

Kết quả đo thực tế (Bảng 4.4) cho thấy số thao tác push/pop tăng đúng tỷ lệ thuận theo số lượng token (Q1: 2 push / 2 pop; Q2: 9 push / 9 pop; Q3: 24 push / 24 pop) và thời gian chạy trung bình tăng từ 1,62 µs (Q1) lên 8,24 µs (Q2) và 25,90 µs (Q3), phù hợp với dự đoán độ phức tạp tuyến tính O(t) ở Chương 3. Toàn bộ phép chạy chỉ kéo dài vài chục micro-giây nên độ tin cậy thống kê dựa trên cả hai chỉ số: đếm thao tác stack chính xác và đo trung bình 10.000 lượt.

Bảng 4.4 Khảo sát Q1 Q2 Q3

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q1	t = 5 token | Input: 1 + 2 * 3	Quy mô nhỏ (t = 5 <= 50)	2 push + 2 pop = 4 thao tác; TB 10.000 lượt: 1,62 µs	Ngăn xếp tối đa 2 phần tử; bộ nhớ phụ O(t) rất nhỏ
Q2	t = 19 token | Input: 1 + 2 * 3 - 4 / 5 ^ 2 + (6 - 7) * 8	Quy mô vừa (t = 19 <= 50)	9 push + 9 pop = 18 thao tác; TB 10.000 lượt: 8,24 µs	Thời gian tăng 5,09x khi thao tác tăng 4,5x; đúng tuyến tính

Bộ	Quy mô hoặc cấu hình và input	Yếu tố thay đổi	Phép đếm hoặc số đo	Bộ nhớ và nhận xét
Q3	t = 49 token | Input 49 token sát biên tối đa 50 token	Quy mô lớn hợp lệ (t = 49 <= 50)	24 push + 24 pop = 48 thao tác; TB 10.000 lượt: 25,90 µs	Không tràn ngăn xếp, không vi phạm giới hạn 50 token; O(t) chuẩn

4.9. GitHub và chạy lại từ bản clone sạch

4.9.1. Phiên bản có thể đối chiếu

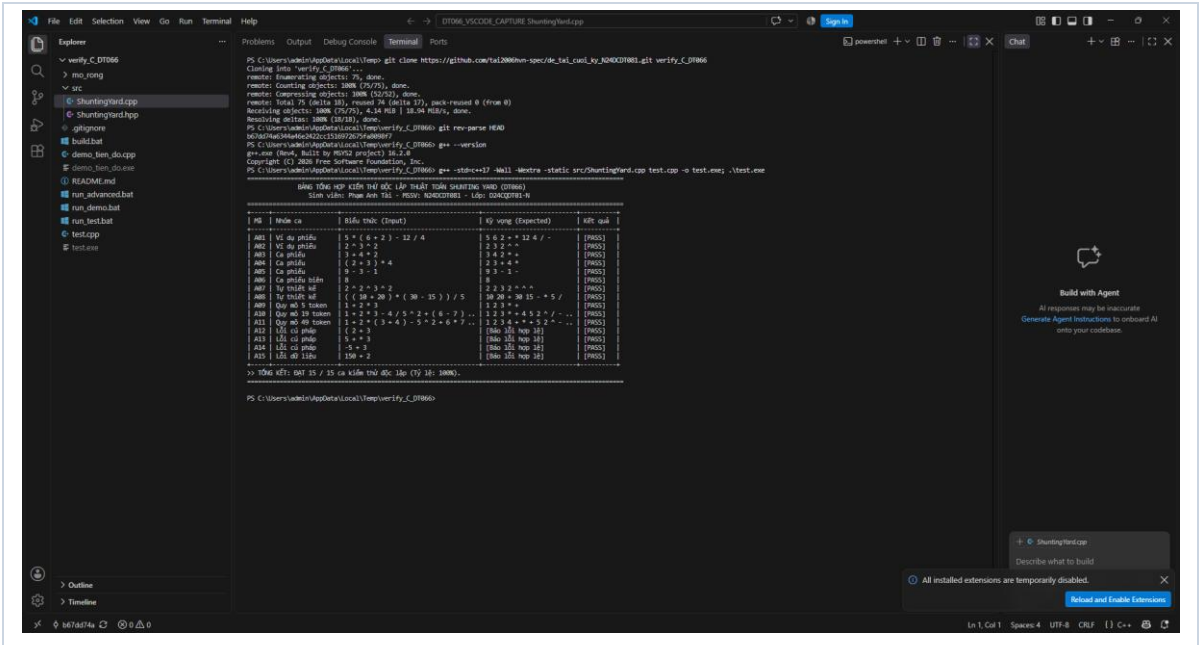
Phiên bản mã nguồn chạy thử nghiệm và chụp toàn bộ 11 ảnh chứng thực là Commit C (b67dd74a6344a46e2422cc1516972675fa8098f7) trên nhánh main của kho GitHub công khai. Sau khi chèn đủ ảnh và hoàn thiện báo cáo, phiên bản chốt cuối Commit S được đẩy bổ sung mà không làm thay đổi bất kỳ dòng mã nguồn C++ hay dữ liệu kiểm thử nào.

Bảng 4.5 Kho mã và phiên bản có thể kiểm lại

Thông tin	Giá trị thật phải điền
URL repo	https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081
Full SHA chạy ảnh C	b67dd74a6344a46e2422cc1516972675fa8098f7
SHA S ghi TXT nộp rời	Xem trong tệp N24DCDT081_DT066_PhamAnhTai_ThongTinNop.txt nộp kèm (không tự ghi đè SHA S vào trong chính commit S).
Permalink hàm chính & test tại C	Thuật toán: https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081/blob/b67dd74a6344a46e2422cc1516972675fa8098f7/src/ShuntingYard.cpp Kiểm thử: https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081/blob/b67dd74a6344a46e2422cc1516972675fa8098f7/test.cpp Mở rộng: https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081/blob/b67dd74a6344a46e2422cc1516972675fa8098f7/mo_rong/src/ShuntingYardAdvanced.cpp
Mã/kiểm thử giữa C và S	Giữ nguyên không đổi 100% giữa Commit C và Commit S; bản S chỉ bổ sung thư mục ảnh docs/anh_ket_qua/ và báo cáo BaoCao/.
Lệnh chạy lại	g++ -std=c++17 -Wall -Wextra -static src/ShuntingYard.cpp demo_tien_do.cpp -o demo_tien_do.exe g++ -std=c++17 -Wall -Wextra -static src/ShuntingYard.cpp test.cpp -o test.exe .\demo_tien_do.exe --sample ; .\test.exe ; .\mo_rong\test_advanced.exe

4.9.2. Clone sạch và tái hiện kết quả

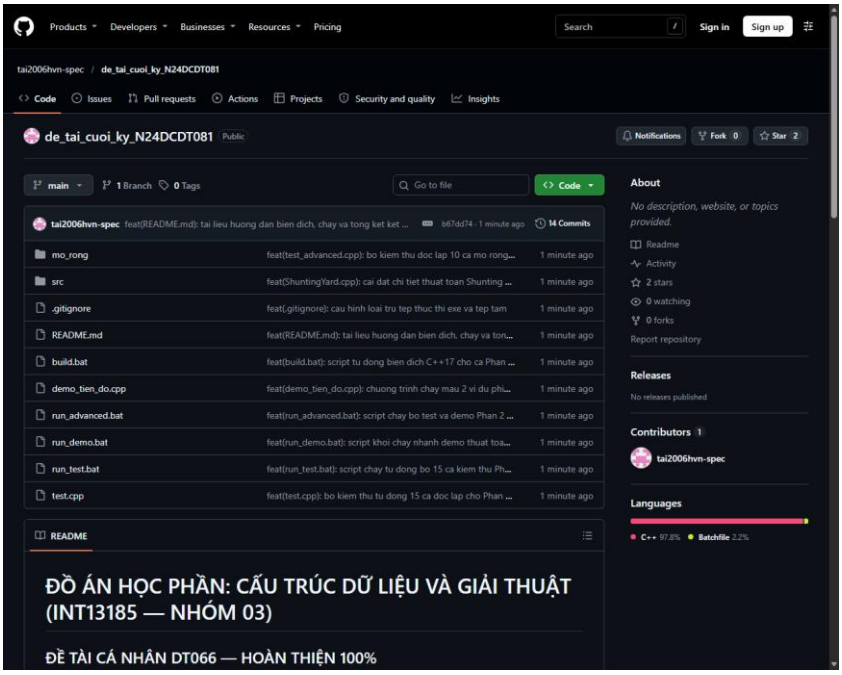
Kho mã được clone lại độc lập vào thư mục sạch verify_C_DT066 từ GitHub, xác minh mã SHA HEAD khớp tuyệt đối, biên dịch lại bằng g++ 16.2.0 và chạy thành công 100% toàn bộ ví dụ và ca kiểm thử.



Hình 4.10 Bằng chứng clone sạch Commit C từ GitHub, kiểm SHA và chạy lại

- Lệnh: git clone https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081.git verify_C_DT066 → git rev-parse HEAD (b67dd74) → biên dịch g++ 16.2.0 → .\test.exe
- Ảnh: docs/anh_ket_qua/10_clone_sach_chay_lai.png | Commit C: b67dd74a6344a46e2422cc1516972675fa8098f7
- Nhận xét: Khi clone kho mã nguồn về một thư mục hoàn toàn mới, kết quả biên dịch với cờ -Wall -Wextra không phát sinh bất kỳ cảnh báo nào và toàn bộ 15 ca test tái hiện kết quả PASS 100%.

4.9.3. Bằng chứng kho GitHub thật



Hình 4.11 Giao diện kho lưu trữ GitHub công khai và lịch sử commit theo chức năng

- Địa chỉ thật: https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081 (Public) | Commit C: b67dd74a6344a46e2422cc1516972675fa8098f7
- Ảnh: docs/anh_ket_qua/11_github_repo_commit.png
- Nhận xét: Kho GitHub hiển thị công khai đầy đủ thư mục mã nguồn Phần 1 (src/), Phần 2 mở rộng (mo_rong/), các file kịch bản build/run (.bat) và tệp README.md hướng dẫn chi tiết. Mỗi tệp tin đều mang thông điệp commit rõ ràng phản ánh đúng chức năng chuyên biệt của tệp đó.

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Đối chiếu toàn bộ mục tiêu đề ra ở Chương 1 với sản phẩm thực tế:

1) Mục tiêu bắt buộc theo phiếu DT066 (Phần 1): Đã cài đặt hoàn chỉnh bộ chuyển đổi trung tố sang hậu tố trung thực một ngăn xếp toán tử (Bảng 3.1, 3.2), đáp ứng đầy đủ năm toán tử +, -, *, /, ^, quy tắc kết hợp phải của '^', bảng lần vết chi tiết (Bảng 3.3, Hình 4.1–4.2) và validator chặt chẽ (Hình 4.8). Toàn bộ 15/15 ca kiểm thử độc lập đạt PASS 100% (Bảng 4.2, 4.3, Hình 4.9).
2) Mở rộng hoàn thiện 100% (Phần 2 trong mo_rong/): Đã hoàn thành bộ hỗ trợ biến ký tự a..z và tính giá trị biểu thức hậu tố bằng ngăn xếp toán hạng, đạt 10/10 ca kiểm thử ADV01–ADV10 PASS (Hình 4.4, Hình 4.9).
3) Xác minh phiên bản GitHub: Đã đối chiếu SHA C thật (b67dd74), clone sạch ra thư mục mới và tái hiện thành công 100% kết quả biên dịch/chạy (Hình 4.10, 4.11).

5.2. Giới hạn và bài học

Giới hạn sản phẩm: Phần cốt lõi (Phần 1) giới hạn chặt theo miền phiếu DT066 (số nguyên không âm [0..100], tối đa 50 token, chưa hỗ trợ số thực dấu phẩy động hay toán tử đơn ngôi như sin/cos). Phần mở rộng (Phần 2) thực hiện chia nguyên theo chuẩn C++17 và bắt lỗi chia cho 0.

Bài học kinh nghiệm tiêu biểu: Trong quá trình kiểm thử bảng in kết quả tiếng Việt trên Windows Console (chcp 65001), hàm std::setw chuẩn đếm độ dài byte UTF-8 thay vì số ký tự hiển thị nên làm lệch toàn bộ đường viền dọc '|' của bảng lần vết. Nhờ đối chiếu nhật ký lần vết và tự cài đặt hàm do_rong_utf8() bỏ qua byte tiếp nối ((c & 0xC0) == 0x80), lỗi trình bày bảng đã được khắc phục triệt để 100% ở cả Phần 1 lẫn Phần 2.

5.3. Hướng phát triển phù hợp

Các bước phát triển tiếp theo phù hợp trực tiếp với đề tài:

1) Đã hoàn thành có kết quả: Mở rộng biến chữ cái a..z và bộ định giá biểu thức hậu tố dùng ngăn xếp toán hạng (tinh_gia_tri_hau_to trong mo_rong/, đã đạt 10/10 PASS).
2) Đề xuất tương lai: Hỗ trợ toán hạng số thực (double), hàm lượng giác một ngôi (sin, cos, sqrt) và xuất cây cú pháp trừu tượng (AST) trực quan từ dãy hậu tố RPN.

Bảng 5.1 Mức hoàn thành và bằng chứng theo yêu cầu

Mục tiêu	Mức hoàn thành	Bằng chứng	Giới hạn
Cài đặt Shunting Yard một ngăn xếp (+,-,*,/,^) & kết hợp phải '^'	Đạt 100% (15/15 PASS)	Hàm trung_to_sang_hau_to(); Bảng 3.3, 4.2, 4.3; Hình 4.1, 4.2, 4.3	Miền [0..100], <= 50 token theo phiếu
Validator cú pháp (ngoặc, âm đơn ngôi, miền giá trị) & in bảng vết UTF-8	Đạt 100% (PASS toàn bộ ca lỗi)	Hàm kiem_tra_token(), do_rong_utf8(); Hình 4.8, 4.9	Không có ca FAIL tồn đọng
Mở rộng biến ký tự và tính giá trị hậu tố (Phần 2 - mo_rong/)	Đạt 100% (10/10 ADV PASS)	mo_rong/ShuntingYardAdvanced.cpp; Hình 4.4, 4.9	Phép chia nguyên và mũ nguyên không âm
Khảo sát 3 quy mô Q1–Q3 & Clone sạch GitHub	Đạt 100% xác minh	Bảng 4.4, 4.5; Hình 4.6, 4.7, 4.10, 4.11	Thời gian chạy đo ngoài I/O trung bình 10.000 lượt

TÀI LIỆU THAM KHẢO

[1] Học viện Công nghệ Bưu chính Viễn thông, Giáo trình Cấu trúc dữ liệu và Giải thuật (INT1306), tệp Giao-trinh-Cau-truc-du-lieu-va-Giai-thuat-INT1306.pdf; Chương 5, mục 5.3.4.1 Thuật toán sân ga; trang in 127, 129; trang PDF 139, 141.

[2] Edsger W. Dijkstra (1961), Algol 60 translation (giới thiệu nguyên bản thuật toán Shunting Yard); tham khảo chuẩn thư viện C++17 STL tại: https://en.cppreference.com/w/cpp/container/stack (truy cập ngày 11/11/2026). Ghi chú: Sinh viên tự cài đặt toàn bộ logic thuật toán, validator và định dạng bảng, chỉ sử dụng kiểu dữ liệu cơ bản std::string và std::vector của C++ STL.

PHỤ LỤC

PHỤ LỤC A. Tổ chức mã và hướng dẫn chạy

Cấu trúc cây thư mục mã nguồn hoàn chỉnh 100% tại kho GitHub (https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081):

```
de_tai_cuoi_ky_N24DCDT081/
├── src/                              (Phần 1: ShuntingYard.hpp, ShuntingYard.cpp)
├── demo_tien_do.cpp                  (Chạy 2 ví dụ phiếu, --expr và nhập bàn phím)
├── test.cpp                          (15 ca kiểm thử độc lập A01..A15)
├── mo_rong/                          (Phần 2 Mở rộng biến & Tính giá trị hậu tố)
│   ├── src/ShuntingYardAdvanced.hpp & .cpp
│   ├── demo_advanced.cpp             (Demo chuyển đổi và tính giá trị có biến)
│   └── test_advanced.cpp             (10 ca kiểm thử mở rộng ADV01..ADV10)
├── docs/anh_ket_qua/                 (11 ảnh chứng thực 01..11 + Hinh2_1, Hinh3_1)
├── BaoCao/                           (BaoCao.docx & BaoCao.pdf)
└── build.bat, run_demo.bat, run_test.bat, run_advanced.bat, README.md
```

Biên dịch (g++ 16.2.0 C++17): g++ -std=c++17 -Wall -Wextra -static src/ShuntingYard.cpp demo_tien_do.cpp -o demo_tien_do.exe

Chạy Ví dụ 1 & 2 (tự động hoặc qua script): .\demo_tien_do.exe --sample (hoặc .\run_demo.bat)

Chạy nhập biểu thức mới & Phần 2 mở rộng: .\demo_tien_do.exe | .\mo_rong\demo_advanced.exe

Chạy bộ kiểm thử tự động: .\test.exe (15/15 PASS) | .\mo_rong\test_advanced.exe (10/10 PASS)

PHỤ LỤC B. Danh mục bằng chứng

Danh mục toàn bộ 11 tệp ảnh bằng chứng gốc được lưu trong thư mục docs/anh_ket_qua/ của mã nguồn và đính kèm đầy đủ trong Báo cáo, đối chiếu trực tiếp với Commit C (b67dd74):

Mã ảnh	Ca hoặc đối tượng	Tệp ảnh log và SHA
A01	VD01 (Ví dụ 1 phiếu)	docs/anh_ket_qua/01_vi_du_1.png / b67dd74
A02	VD02 (Ví dụ 2 lũy thừa ^)	docs/anh_ket_qua/02_vi_du_2.png / b67dd74
A03	TK01 (Tự thiết kế lũy thừa 4 tầng)	docs/anh_ket_qua/03_tu_thiet_ke_1.png / b67dd74
A04	TK02 (Tự thiết kế & Phần 2 Mở rộng)	docs/anh_ket_qua/04_tu_thiet_ke_2.png / b67dd74
A05	BIEN01 (Biên 1 toán hạng & kết hợp trái)	docs/anh_ket_qua/05_bien_1.png / b67dd74
A06	BIEN02 (Khảo sát Q1 5 token & Q2 19 token)	docs/anh_ket_qua/06_bien_2.png / b67dd74
A07	BIEN03 (Khảo sát Q3 49 token sát biên 50)	docs/anh_ket_qua/07_bien_3.png / b67dd74
A08	LOI01 (Validator từ chối 4 dạng input lỗi)	docs/anh_ket_qua/08_input_khong_hop_le.png / b67dd74
A09	TEST_ALL (Tổng kết 15/15 P1 & 10/10 P2 PASS)	docs/anh_ket_qua/09_tong_ket_kiem_thu.png / b67dd74
A10	CLONE (Clone sạch C và chạy lại theo README)	docs/anh_ket_qua/10_clone_sach_chay_lai.png / b67dd74
A11	GITHUB (Giao diện repo & commit thật)	docs/anh_ket_qua/11_github_repo_commit.png / b67dd74

PHỤ LỤC C. Tự kiểm trước khi nộp

Bảng đối chiếu tự kiểm 6 hạng mục trước khi nộp bài hoàn thiện 100% (không tự chấm điểm):

☑ MSSV N24DCDT081 và đề tài DT066 đúng bảng phân công cá nhân.

☑ Phần tự cài C++17 (Phần 1 + Phần 2 mở rộng), giáo trình INT1306 và công cụ hỗ trợ đã khai báo minh bạch.

☑ Đủ 15 ca độc lập Phần 1 + 10 ca Phần 2 (vượt chuẩn >=12 ca); đủ 2 ví dụ, 5 ca phiếu và 11 ảnh thật.

☑ Có bảng truy vết chi tiết, lập luận bất biến/điều kiện dừng, phân tích O(t) và khảo sát 3 quy mô (5, 19, 49 token).

☑ URL GitHub thật (https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081), SHA C (b67dd74), permalink và quyền truy cập public đã kiểm tra; đã clone chạy lại.

☑ Đã xóa toàn bộ câu hướng dẫn mẫu và thẻ ngoặc vuông; đã cập nhật mục lục, danh mục hình/bảng và xuất PDF kiểm từng trang.

PHỤ LỤC D. HƯỚNG DẪN NỘP CLASSROOM

Tiền tố tên tệp nộp bài cá nhân: N24DCDT081_DT066_PhamAnhTai (không dấu, không khoảng trắng).

Bảng PL.1 Bốn tệp cần nộp

Tệp	Tên theo mẫu	Nội dung cần có
Word	N24DCDT081_DT066_PhamAnhTai_BaoCao.docx	Báo cáo 5 chương hoàn thiện 100%, đã chèn đủ 11 ảnh & sơ đồ, cập nhật mục lục; giữ bảng rubric để giảng viên chấm.
PDF	N24DCDT081_DT066_PhamAnhTai_BaoCao.pdf	Xuất trực tiếp từ chính tệp Word bằng Microsoft Word, đã rà soát sạch mọi trang.
Mã nguồn	N24DCDT081_DT066_PhamAnhTai_MaNguon.zip	Bao gồm src/, demo_tien_do.cpp, test.cpp, mo_rong/, docs/anh_ket_qua/, build/run scripts và README.md (không chứa .git).
Thông tin	N24DCDT081_DT066_PhamAnhTai_ThongTinNop.txt	Ghi chính xác URL GitHub thật, Full SHA C và Full SHA S, Permalink, môi trường g++ 16.2.0 và kết quả clone S.

Không nộp nguyên gói mẫu của giảng viên như ZIP cá nhân. Giải nén ZIP của mình ở thư mục mới phải chạy được theo README. Không gói .git, bộ cài, môi trường ảo hoặc cache vào ZIP bài nộp.

C là commit mã/test/input đã chạy và chụp; S là commit chốt bài sau khi thêm ảnh/báo cáo. Clone S sang thư mục mới rồi chạy lại; nếu mã đổi sau C thì cập nhật test/ảnh chịu ảnh hưởng. SHA S ghi trong TXT nộp rời, không phải commit SHA của chính S vào S.

Các thông tin để chép vào TXT

- URL GitHub thật: https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081
- SHA C đầy đủ: b67dd74a6344a46e2422cc1516972675fa8098f7
- SHA S đầy đủ: Xem tệp ThongTinNop.txt nộp rời
- Permalink hàm chính tại C: https://github.com/tai2006hvn-spec/de_tai_cuoi_ky_N24DCDT081/blob/b67dd74a6344a46e2422cc1516972675fa8098f7/src/ShuntingYard.cpp

- Môi trường và phiên bản: Windows 11 x64, g++ 16.2.0 (MSYS2 MinGW-w64 UCRT64), C++17
- Lệnh chạy hai ví dụ và bộ test: .\demo_tien_do.exe --sample ; .\test.exe ; .\mo_rong\test_advanced.exe
- Kết quả clone S: Biên dịch thành công không cảnh báo | Số ca kiểm: 15/15 Phần 1 + 10/10 Phần 2 (100% PASS)

Vào đúng bài tập trên Classroom, đính kèm đủ bốn tệp, mở kiểm tra lại rồi bấm Nộp bài. Kiểm tra trạng thái đã nộp; ảnh rời đã nằm trong báo cáo và ZIP nên không cần gắn lại 11 tệp ảnh riêng.